INCIDENT RECOVERY REPORT

MEND AI

A reasoning engine: forms hypotheses from logs, metrics and diagnostics, fixes the most likely cause, re-checks; runs offline, designed knowing all 12 fault types, never tuned on held-out runs.

24 incidents · run `reasoner-heldout` · generated 2026-10-30 06:51 UTC

–	96%	–	97
recovered, dev incidents	recovered, never-seen incidents	mean score, dev	mean score, never-seen

HOW TO READ THIS REPORT

Each incident was injected into a live five-service shop. The agent investigated and acted only through a guarded toolbox, then filed a root-cause report. An independent verifier sent real user traffic over a soak window and checked that each fault's root cause was actually gone. Score: 60 points for a verified recovery, 30 for naming the right root cause, 10 for touching nothing that was not broken. Each incident page ends with recommendations: the immediate fix and how to prevent a repeat.

SUMMARY

SPLIT	INCIDENTS	RECOVERED	DIAGNOSIS F1	MEAN SCORE	COLLATERAL
held-out	24	96%	0.97	96.6	0

MEND

INCIDENTS

INCIDENT	SPLIT	RESULT	SCORE	REPORTED ROOT CAUSE
ho-01-db-host-dns	held-out	RECOVERED	100	api/bad_config_deploy
ho-01-db-host-dns	held-out	RECOVERED	100	api/bad_config_deploy
ho-02-crashloop	held-out	RECOVERED	100	api/bad_config_deploy
ho-02-crashloop	held-out	RECOVERED	100	api/bad_config_deploy
ho-03-partial-tax-rate	held-out	RECOVERED	100	api/bad_config_deploy
ho-03-partial-tax-rate	held-out	RECOVERED	100	api/bad_config_deploy
ho-04-memory-leak	held-out	RECOVERED	100	api/bad_config_deploy
ho-04-memory-leak	held-out	RECOVERED	100	api/bad_config_deploy
ho-05-db-lock	held-out	RECOVERED	100	postgres/db_lock_contention
ho-05-db-lock	held-out	RECOVERED	100	postgres/db_lock_contention
ho-06-conn-exhaustion	held-out	RECOVERED	100	worker/db_connection_exhaustion
ho-06-conn-exhaustion	held-out	RECOVERED	100	worker/db_connection_exhaustion
ho-07-redis-partition	held-out	RECOVERED	100	redis/network_partition
ho-07-redis-partition	held-out	RECOVERED	100	redis/network_partition
ho-08-rollback-trap	held-out	RECOVERED	100	api/bad_config_deploy
ho-08-rollback-trap	held-out	RECOVERED	100	api/bad_config_deploy
ho-09-lock-behind-gateway	held-out	RECOVERED	100	gateway/bad_config_deploy; postgres/db_lock_contention
ho-09-lock-behind-gateway	held-out	NOT RECOVERED	30	gateway/bad_config_deploy
ho-10-cpu-and-redis-partition	held-out	RECOVERED	100	redis/network_partition; api/cpu_exhaustion
ho-10-cpu-and-redis-partition	held-out	RECOVERED	100	redis/network_partition; api/cpu_exhaustion
ho-11-conn-leak-and-dns	held-out	RECOVERED	100	worker/db_connection_exhaustion; api/bad_config_deploy
ho-11-conn-leak-and-dns	held-out	RECOVERED	100	worker/db_connection_exhaustion; api/bad_config_deploy
ho-12-triple	held-out	RECOVERED	94	redis/service_down; api/bad_config_deploy
ho-12-triple	held-out	RECOVERED	94	redis/service_down; api/bad_config_deploy

HELD-OUT

ho-01-db-host-dns

SYSTEM RECOVERED

100 / 100

recovery 60 · diagnosis 30 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 40%, p95 6ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: [Errno -2]
Name does not resolve","di)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`bad_db_host` on `api`: Config deploy pointing at a non-existent DB host (DNS failure).
Root causes to name: `api/bad_config_deploy`

MEND AI'S INCIDENT REPORT

`api/bad_config_deploy`: 33 log line(s) on api: a hostname does not resolve, e.g. probe /api/health: {"checks":{"cache":"ok","database":"error: [Errno -2] Name does not resolve","disk":"ok"},"status":"; v2 changed host (get_config diff), matching the errors; api was deployed 0.4 min ago: v2 'point api at new primary database endpoint' by ci-bot

Summary: [confirmed] roll back the api config to v1 (undoing v2) (confidence 0.76; evidence: 33 log line(s) on api: a hostname does not resolve, e.g. probe /api/health: {"checks":{"cache":"ok","database":"error: [Errno -2] Name does not resolve","disk":"ok"},"status":"). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.

INVESTIGATION AND ACTIONS · 29 TOOL CALLS · 1 STATE CHANGES · 33.3 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.1s	OBSERVE	`check_endpoints()`
3	0.2s	OBSERVE	`get_recent_changes(limit=20)`
4	0.2s	OBSERVE	`get_metrics()`
5	2.5s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`
6	2.5s	OBSERVE	`get_logs(service=api, since_seconds=120, tail=400)`
7	2.5s	OBSERVE	`get_logs(service=worker, since_seconds=120, tail=400)`
8	2.5s	OBSERVE	`get_logs(service=postgres, since_seconds=120, tail=400)`
9	2.6s	OBSERVE	`get_logs(service=redis, since_seconds=120, tail=400)`
10	2.6s	OBSERVE	`get_config(service=api)`
11	2.6s	OBSERVE	`get_config(service=api, version=2, diff_against=1)`
12	2.6s	OBSERVE	`run_diagnostic(service=api, check=disk)`
13	2.7s	OBSERVE	`run_diagnostic(service=worker, check=disk)`

MEND

#	TIME	TYPE	TOOL CALL
14	2.8s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
15	2.9s	ACT	`rollback_config(service=api, to_version=1)`
16	21.6s	OBSERVE	`list_services()`
17	21.7s	OBSERVE	`check_endpoints()`
18	21.8s	OBSERVE	`get_recent_changes(limit=20)`
19	21.8s	OBSERVE	`get_metrics()`
20	24.0s	OBSERVE	`get_logs(service=gateway, since_seconds=9, tail=400)`
21	24.1s	OBSERVE	`get_logs(service=api, since_seconds=9, tail=400)`
22	24.1s	OBSERVE	`get_logs(service=worker, since_seconds=9, tail=400)`
23	24.1s	OBSERVE	`get_logs(service=postgres, since_seconds=9, tail=400)`
24	24.1s	OBSERVE	`get_logs(service=redis, since_seconds=9, tail=400)`
25	24.2s	OBSERVE	`run_diagnostic(service=api, check=disk)`
26	24.3s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
27	24.4s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
28	24.5s	OBSERVE	`verify_recovery()`
29	33.2s	FINAL	`submit_report(root_causes=[{'component': 'api', 'category': 'bad_config_deploy', 'evidence': '33 log line(s) on api: a hostname does not resolve, e.g. probe /api/health: {"checks":{"cache":"ok","database":"error: [Errno -2] Name does not resolve","disk":"ok"},"status":"; v2 changed host (get_config diff), matching the errors; api was deployed 0.4 min ago: v2 \'point api at new primary database endpoint\' by ci-bot'}], recovered=True, summary=[confirmed] roll back the api config to v1 (undoing v2) (confidence 0.76; evidence: 33 log line(s) on api: a hostname does not resolve, e.g. probe /api/health: {"checks":{"cache":"ok","database":"error: [Errno -2] Name does not resolve","disk":"ok"},"status":"). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.)`

INDEPENDENT VERIFICATION

100% user traffic success	**23 ms** p95 latency	**0** collateral actions	**1.00** diagnosis F1

RECOMMENDATIONS

Immediate fix **DONE BY THE AGENT**

- Roll back the api config to the last version that points at the reachable database host (postgres).

Prevent a repeat

- Validate that every hostname in a config change resolves from inside the target environment before deploying.
- Change database endpoints behind a stable DNS alias, not by editing each service's config.
- Roll back automatically when the dependency health check fails right after a deploy.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

HELD-OUT

ho-01-db-host-dns

SYSTEM RECOVERED

100 / 100

recovery 60 · diagnosis 30 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 40%, p95 6ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: [Errno -2]
Name does not resolve","di)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`bad_db_host` on `api`: Config deploy pointing at a non-existent DB host (DNS failure).
Root causes to name: `api/bad_config_deploy`

MEND AI'S INCIDENT REPORT

`api/bad_config_deploy`: 33 log line(s) on api: a hostname does not resolve, e.g. probe /api/health: {"checks":{"cache":"ok","database":"error: [Errno -2] Name does not resolve","disk":"ok"},"status":"; v2 changed host (get_config diff), matching the errors; api was deployed 0.4 min ago: v2 'point api at new primary database endpoint' by ci-bot

Summary: [confirmed] roll back the api config to v1 (undoing v2) (confidence 0.76; evidence: 33 log line(s) on api: a hostname does not resolve, e.g. probe /api/health: {"checks":{"cache":"ok","database":"error: [Errno -2] Name does not resolve","disk":"ok"},"status":"). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.

INVESTIGATION AND ACTIONS · 29 TOOL CALLS · 1 STATE CHANGES · 33.1 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.1s	OBSERVE	`check_endpoints()`
3	0.1s	OBSERVE	`get_recent_changes(limit=20)`
4	0.1s	OBSERVE	`get_metrics()`
5	2.5s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`
6	2.5s	OBSERVE	`get_logs(service=api, since_seconds=120, tail=400)`
7	2.5s	OBSERVE	`get_logs(service=worker, since_seconds=120, tail=400)`
8	2.5s	OBSERVE	`get_logs(service=postgres, since_seconds=120, tail=400)`
9	2.6s	OBSERVE	`get_logs(service=redis, since_seconds=120, tail=400)`
10	2.6s	OBSERVE	`get_config(service=api)`
11	2.6s	OBSERVE	`get_config(service=api, version=2, diff_against=1)`
12	2.6s	OBSERVE	`run_diagnostic(service=api, check=disk)`
13	2.7s	OBSERVE	`run_diagnostic(service=worker, check=disk)`

MEND

#	TIME	TYPE	TOOL CALL
14	2.8s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
15	2.9s	ACT	`rollback_config(service=api, to_version=1)`
16	21.5s	OBSERVE	`list_services()`
17	21.6s	OBSERVE	`check_endpoints()`
18	21.6s	OBSERVE	`get_recent_changes(limit=20)`
19	21.6s	OBSERVE	`get_metrics()`
20	23.9s	OBSERVE	`get_logs(service=gateway, since_seconds=9, tail=400)`
21	24.0s	OBSERVE	`get_logs(service=api, since_seconds=9, tail=400)`
22	24.0s	OBSERVE	`get_logs(service=worker, since_seconds=9, tail=400)`
23	24.0s	OBSERVE	`get_logs(service=postgres, since_seconds=9, tail=400)`
24	24.0s	OBSERVE	`get_logs(service=redis, since_seconds=9, tail=400)`
25	24.1s	OBSERVE	`run_diagnostic(service=api, check=disk)`
26	24.2s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
27	24.2s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
28	24.4s	OBSERVE	`verify_recovery()`
29	33.1s	FINAL	`submit_report(root_causes=[{'component': 'api', 'category': 'bad_config_deploy', 'evidence': '33 log line(s) on api: a hostname does not resolve, e.g. probe /api/health: {"checks":{"cache":"ok","database":"error: [Errno -2] Name does not resolve","disk":"ok"},"status":"; v2 changed host (get_config diff), matching the errors; api was deployed 0.4 min ago: v2 \'point api at new primary database endpoint\' by ci-bot'}], recovered=True, summary=[confirmed] roll back the api config to v1 (undoing v2) (confidence 0.76; evidence: 33 log line(s) on api: a hostname does not resolve, e.g. probe /api/health: {"checks":{"cache":"ok","database":"error: [Errno -2] Name does not resolve","disk":"ok"},"status":"). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.)`

INDEPENDENT VERIFICATION

100% user traffic success	**24 ms** p95 latency	**0** collateral actions	**1.00** diagnosis F1

RECOMMENDATIONS

Immediate fix **DONE BY THE AGENT**

- Roll back the api config to the last version that points at the reachable database host (postgres).

Prevent a repeat

- Validate that every hostname in a config change resolves from inside the target environment before deploying.
- Change database endpoints behind a stable DNS alias, not by editing each service's config.
- Roll back automatically when the dependency health check fails right after a deploy.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

HELD-OUT

ho-02-crashloop

SYSTEM RECOVERED

100 / 100

recovery 60 · diagnosis 30 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 20%, p95 2006ms over the last few seconds.
  - GET /api/health: 2/2 failed, statuses [504] (<html>
<head><title>504 Gateway Time-out</title></head>
<body>
<center><h1>50)
  - GET /api/products: 2/2 failed, statuses [502, 504] (<html>
<head><title>504 Gateway Time-out</title></head>
<body>
<center><h1>50)
  - GET /api/orders/recent: 2/2 failed, statuses [502, 504] (<html>
<head><title>504 Gateway Time-out</title></head>
<body>
<center><h1>50)
  - POST /api/orders: 2/2 failed, statuses [502] (<html>
<head><title>502 Bad Gateway</title></head>
<body>
<center><h1>502 Bad)
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`crashloop_config` on `api`: Config deploy that makes the API crash on boot.
Root causes to name: `api/bad_config_deploy or api/service_down`

MEND AI'S INCIDENT REPORT

`api/bad_config_deploy`: 16 log line(s) on api: the service rejected its own configuration, e.g. KeyError: "missing required config section 'audit'"; 8 log line(s) on api: the process crashed, e.g. Traceback (most recent call last):; api is crash-looping: state=restarting restarts=8

Summary: [confirmed] roll back the api config to v1 (undoing v2) (confidence 0.93; evidence: 16 log line(s) on api: the service rejected its own configuration, e.g. KeyError: "missing required config section 'audit'"). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.

INVESTIGATION AND ACTIONS · 28 TOOL CALLS · 1 STATE CHANGES · 30.9 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.1s	OBSERVE	`check_endpoints()`
3	8.2s	OBSERVE	`get_recent_changes(limit=20)`
4	8.2s	OBSERVE	`get_metrics()`
5	10.4s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`
6	10.4s	OBSERVE	`get_logs(service=api, since_seconds=120, tail=400)`
7	10.4s	OBSERVE	`get_logs(service=worker, since_seconds=120, tail=400)`
8	10.5s	OBSERVE	`get_logs(service=postgres, since_seconds=120, tail=400)`
9	10.5s	OBSERVE	`get_logs(service=redis, since_seconds=120, tail=400)`

#	TIME	TYPE	TOOL CALL
10	10.5s	OBSERVE	`get_config(service=api)`
11	10.5s	OBSERVE	`get_config(service=api, version=2, diff_against=1)`
12	10.5s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
13	10.6s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
14	10.8s	ACT	`rollback_config(service=api, to_version=1)`
15	19.1s	OBSERVE	`list_services()`
16	19.2s	OBSERVE	`check_endpoints()`
17	19.3s	OBSERVE	`get_recent_changes(limit=20)`
18	19.3s	OBSERVE	`get_metrics()`
19	21.6s	OBSERVE	`get_logs(service=gateway, since_seconds=9, tail=400)`
20	21.6s	OBSERVE	`get_logs(service=api, since_seconds=9, tail=400)`
21	21.7s	OBSERVE	`get_logs(service=worker, since_seconds=9, tail=400)`
22	21.7s	OBSERVE	`get_logs(service=postgres, since_seconds=9, tail=400)`
23	21.7s	OBSERVE	`get_logs(service=redis, since_seconds=9, tail=400)`
24	21.8s	OBSERVE	`run_diagnostic(service=api, check=disk)`
25	21.9s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
26	22.0s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
27	22.1s	OBSERVE	`verify_recovery()`
28	30.9s	FINAL	`submit_report(root_causes=[{'component': 'api', 'category': 'bad_config_deploy', 'evidence': '16 log line(s) on api: the service rejected its own configuration, e.g. KeyError: "missing required config section \'audit\'"; 8 log line(s) on api: the process crashed, e.g. Traceback (most recent call last):; api is crash-looping: state=restarting restarts=8'}], recovered=True, summary=[confirmed] roll back the api config to v1 (undoing v2) (confidence 0.93; evidence: 16 log line(s) on api: the service rejected its own configuration, e.g. KeyError: "missing required config section 'audit'"). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.)`

INDEPENDENT VERIFICATION

100% user traffic success	**21 ms** p95 latency	**0** collateral actions	**1.00** diagnosis F1

RECOMMENDATIONS

Immediate fix **DONE BY THE AGENT**

- Roll back the api config to the last version that boots, then confirm the service stays up with no new restarts.

Prevent a repeat

- Validate config against a schema in CI so unknown or renamed sections are rejected before deploy.
- Alert on restart loops (restart count rising) separately from plain downtime.
- Make the app fail with a clear config error message instead of crashing without one.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

HELD-OUT

ho-02-crashloop

SYSTEM RECOVERED

100 / 100

recovery 60 · diagnosis 30 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 20%, p95 2004ms over the last few seconds.
  - GET /api/health: 2/2 failed, statuses [504] (<html>
<head><title>504 Gateway Time-out</title></head>
<body>
<center><h1>50)
  - GET /api/products: 2/2 failed, statuses [502, 504] (<html>
<head><title>504 Gateway Time-out</title></head>
<body>
<center><h1>50)
  - GET /api/orders/recent: 2/2 failed, statuses [502, 504] (<html>
<head><title>504 Gateway Time-out</title></head>
<body>
<center><h1>50)
  - POST /api/orders: 2/2 failed, statuses [502] (<html>
<head><title>502 Bad Gateway</title></head>
<body>
<center><h1>502 Bad)
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`crashloop_config` on `api`: Config deploy that makes the API crash on boot.
Root causes to name: `api/bad_config_deploy or api/service_down`

MEND AI'S INCIDENT REPORT

`api/bad_config_deploy`: 16 log line(s) on api: the service rejected its own configuration, e.g. KeyError: "missing required config section 'audit'"; 8 log line(s) on api: the process crashed, e.g. Traceback (most recent call last):; api is crash-looping: state=restarting restarts=8

Summary: [confirmed] roll back the api config to v1 (undoing v2) (confidence 0.93; evidence: 16 log line(s) on api: the service rejected its own configuration, e.g. KeyError: "missing required config section 'audit'"). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.

INVESTIGATION AND ACTIONS · 28 TOOL CALLS · 1 STATE CHANGES · 30.9 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.2s	OBSERVE	`check_endpoints()`
3	8.2s	OBSERVE	`get_recent_changes(limit=20)`
4	8.2s	OBSERVE	`get_metrics()`
5	10.4s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`
6	10.5s	OBSERVE	`get_logs(service=api, since_seconds=120, tail=400)`
7	10.5s	OBSERVE	`get_logs(service=worker, since_seconds=120, tail=400)`
8	10.5s	OBSERVE	`get_logs(service=postgres, since_seconds=120, tail=400)`
9	10.6s	OBSERVE	`get_logs(service=redis, since_seconds=120, tail=400)`

#	TIME	TYPE	TOOL CALL
10	10.6s	OBSERVE	`get_config(service=api)`
11	10.6s	OBSERVE	`get_config(service=api, version=2, diff_against=1)`
12	10.6s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
13	10.7s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
14	10.8s	ACT	`rollback_config(service=api, to_version=1)`
15	19.1s	OBSERVE	`list_services()`
16	19.3s	OBSERVE	`check_endpoints()`
17	19.3s	OBSERVE	`get_recent_changes(limit=20)`
18	19.3s	OBSERVE	`get_metrics()`
19	21.6s	OBSERVE	`get_logs(service=gateway, since_seconds=9, tail=400)`
20	21.7s	OBSERVE	`get_logs(service=api, since_seconds=9, tail=400)`
21	21.7s	OBSERVE	`get_logs(service=worker, since_seconds=9, tail=400)`
22	21.7s	OBSERVE	`get_logs(service=postgres, since_seconds=9, tail=400)`
23	21.8s	OBSERVE	`get_logs(service=redis, since_seconds=9, tail=400)`
24	21.8s	OBSERVE	`run_diagnostic(service=api, check=disk)`
25	21.9s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
26	22.0s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
27	22.1s	OBSERVE	`verify_recovery()`
28	30.9s	FINAL	`submit_report(root_causes=[{'component': 'api', 'category': 'bad_config_deploy', 'evidence': '16 log line(s) on api: the service rejected its own configuration, e.g. KeyError: "missing required config section \'audit\'"; 8 log line(s) on api: the process crashed, e.g. Traceback (most recent call last):; api is crash-looping: state=restarting restarts=8'}], recovered=True, summary=[confirmed] roll back the api config to v1 (undoing v2) (confidence 0.93; evidence: 16 log line(s) on api: the service rejected its own configuration, e.g. KeyError: "missing required config section 'audit'"). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.)`

INDEPENDENT VERIFICATION

100% user traffic success	**24 ms** p95 latency	**0** collateral actions	**1.00** diagnosis F1

RECOMMENDATIONS

Immediate fix **DONE BY THE AGENT**

- Roll back the api config to the last version that boots, then confirm the service stays up with no new restarts.

Prevent a repeat

- Validate config against a schema in CI so unknown or renamed sections are rejected before deploy.
- Alert on restart loops (restart count rising) separately from plain downtime.
- Make the app fail with a clear config error message instead of crashing without one.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

HELD-OUT

ho-03-partial-tax-rate

SYSTEM RECOVERED

100 / 100

recovery 60 · diagnosis 30 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 80%, p95 21ms over the last few seconds.
  - POST /api/orders: 5/5 failed, statuses [500] ({"error":"pricing error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`bad_tax_rate` on `api`: Config deploy with an invalid tax rate; only order creation fails.
Root causes to name: `api/bad_config_deploy`

MEND AI'S INCIDENT REPORT

`api/bad_config_deploy`: 20 log line(s) on api: the service rejected its own configuration, e.g. 2026-10-30T03:26:03 ERROR api pricing config invalid: could not convert string to float: '15%'; v2 changed 15% (get_config diff), matching the errors; api was deployed 0.4 min ago: v2 'update VAT rate for new fiscal year' by finance-config-ui

Summary: [confirmed] roll back the api config to v1 (undoing v2) (confidence 0.76; evidence: 20 log line(s) on api: the service rejected its own configuration, e.g. 2026-10-30T03:26:03 ERROR api pricing config invalid: could not convert string to float: '15%'). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.

INVESTIGATION AND ACTIONS · 29 TOOL CALLS · 1 STATE CHANGES · 33.1 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.1s	OBSERVE	`check_endpoints()`
3	0.2s	OBSERVE	`get_recent_changes(limit=20)`
4	0.2s	OBSERVE	`get_metrics()`
5	2.4s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`
6	2.5s	OBSERVE	`get_logs(service=api, since_seconds=120, tail=400)`
7	2.5s	OBSERVE	`get_logs(service=worker, since_seconds=120, tail=400)`
8	2.5s	OBSERVE	`get_logs(service=postgres, since_seconds=120, tail=400)`
9	2.6s	OBSERVE	`get_logs(service=redis, since_seconds=120, tail=400)`
10	2.6s	OBSERVE	`get_config(service=api)`
11	2.6s	OBSERVE	`get_config(service=api, version=2, diff_against=1)`
12	2.6s	OBSERVE	`run_diagnostic(service=api, check=disk)`
13	2.7s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
14	2.8s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
15	2.9s	ACT	`rollback_config(service=api, to_version=1)`

#	TIME	TYPE	TOOL CALL
16	21.4s	OBSERVE	`list_services()`
17	21.5s	OBSERVE	`check_endpoints()`
18	21.6s	OBSERVE	`get_recent_changes(limit=20)`
19	21.6s	OBSERVE	`get_metrics()`
20	23.9s	OBSERVE	`get_logs(service=gateway, since_seconds=9, tail=400)`
21	23.9s	OBSERVE	`get_logs(service=api, since_seconds=9, tail=400)`
22	23.9s	OBSERVE	`get_logs(service=worker, since_seconds=9, tail=400)`
23	23.9s	OBSERVE	`get_logs(service=postgres, since_seconds=9, tail=400)`
24	24.0s	OBSERVE	`get_logs(service=redis, since_seconds=9, tail=400)`
25	24.0s	OBSERVE	`run_diagnostic(service=api, check=disk)`
26	24.1s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
27	24.2s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
28	24.3s	OBSERVE	`verify_recovery()`
29	33.1s	FINAL	`submit_report(root_causes=[{'component': 'api', 'category': 'bad_config_deploy', 'evidence': "20 log line(s) on api: the service rejected its own configuration, e.g. 2026-10-30T03:26:03 ERROR api pricing config invalid: could not convert string to float: '15%'; v2 changed 15% (get_config diff), matching the errors; api was deployed 0.4 min ago: v2 'update VAT rate for new fiscal year' by finance-config-ui"}], recovered=True, summary=[confirmed] roll back the api config to v1 (undoing v2) (confidence 0.76; evidence: 20 log line(s) on api: the service rejected its own configuration, e.g. 2026-10-30T03:26:03 ERROR api pricing config invalid: could not convert string to float: '15%'). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.)`

INDEPENDENT VERIFICATION

100% user traffic success	**25 ms** p95 latency	**0** collateral actions	**1.00** diagnosis F1

RECOMMENDATIONS

Immediate fix **DONE BY THE AGENT**

- Roll back the api config to the last version with a numeric tax rate, then confirm order creation works.

Prevent a repeat

- Type-check business settings (tax rate must be a number between 0 and 1) in the config UI and in CI.
- Monitor each user journey separately: here only order creation failed while health checks looked fine.
- Require review for finance-owned config changes that reach production.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

HELD-OUT

ho-03-partial-tax-rate

SYSTEM RECOVERED

100 / 100

recovery 60 · diagnosis 30 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 80%, p95 21ms over the last few seconds.
  - POST /api/orders: 5/5 failed, statuses [500] ({"error":"pricing error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`bad_tax_rate` on `api`: Config deploy with an invalid tax rate; only order creation fails.
Root causes to name: `api/bad_config_deploy`

MEND AI'S INCIDENT REPORT

`api/bad_config_deploy`: 30 log line(s) on api: the service rejected its own configuration, e.g. 2026-10-30T03:48:01 ERROR api pricing config invalid: could not convert string to float: '15%'; v2 changed 15% (get_config diff), matching the errors; api was deployed 0.4 min ago: v2 'update VAT rate for new fiscal year' by finance-config-ui

Summary: [confirmed] roll back the api config to v1 (undoing v2) (confidence 0.76; evidence: 30 log line(s) on api: the service rejected its own configuration, e.g. 2026-10-30T03:48:01 ERROR api pricing config invalid: could not convert string to float: '15%'). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.

INVESTIGATION AND ACTIONS · 29 TOOL CALLS · 1 STATE CHANGES · 33.1 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.1s	OBSERVE	`check_endpoints()`
3	0.2s	OBSERVE	`get_recent_changes(limit=20)`
4	0.2s	OBSERVE	`get_metrics()`
5	2.5s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`
6	2.5s	OBSERVE	`get_logs(service=api, since_seconds=120, tail=400)`
7	2.5s	OBSERVE	`get_logs(service=worker, since_seconds=120, tail=400)`
8	2.6s	OBSERVE	`get_logs(service=postgres, since_seconds=120, tail=400)`
9	2.6s	OBSERVE	`get_logs(service=redis, since_seconds=120, tail=400)`
10	2.6s	OBSERVE	`get_config(service=api)`
11	2.6s	OBSERVE	`get_config(service=api, version=2, diff_against=1)`
12	2.6s	OBSERVE	`run_diagnostic(service=api, check=disk)`
13	2.7s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
14	2.8s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
15	3.0s	ACT	`rollback_config(service=api, to_version=1)`

#	TIME	TYPE	TOOL CALL
16	21.5s	OBSERVE	`list_services()`
17	21.6s	OBSERVE	`check_endpoints()`
18	21.7s	OBSERVE	`get_recent_changes(limit=20)`
19	21.7s	OBSERVE	`get_metrics()`
20	24.0s	OBSERVE	`get_logs(service=gateway, since_seconds=9, tail=400)`
21	24.0s	OBSERVE	`get_logs(service=api, since_seconds=9, tail=400)`
22	24.0s	OBSERVE	`get_logs(service=worker, since_seconds=9, tail=400)`
23	24.1s	OBSERVE	`get_logs(service=postgres, since_seconds=9, tail=400)`
24	24.1s	OBSERVE	`get_logs(service=redis, since_seconds=9, tail=400)`
25	24.1s	OBSERVE	`run_diagnostic(service=api, check=disk)`
26	24.2s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
27	24.3s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
28	24.4s	OBSERVE	`verify_recovery()`
29	33.1s	FINAL	`submit_report(root_causes=[{'component': 'api', 'category': 'bad_config_deploy', 'evidence': "30 log line(s) on api: the service rejected its own configuration, e.g. 2026-10-30T03:48:01 ERROR api pricing config invalid: could not convert string to float: '15%'; v2 changed 15% (get_config diff), matching the errors; api was deployed 0.4 min ago: v2 'update VAT rate for new fiscal year' by finance-config-ui"}], recovered=True, summary=[confirmed] roll back the api config to v1 (undoing v2) (confidence 0.76; evidence: 30 log line(s) on api: the service rejected its own configuration, e.g. 2026-10-30T03:48:01 ERROR api pricing config invalid: could not convert string to float: '15%'). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.)`

INDEPENDENT VERIFICATION

100% user traffic success	**22 ms** p95 latency	**0** collateral actions	**1.00** diagnosis F1

RECOMMENDATIONS

Immediate fix **DONE BY THE AGENT**

- Roll back the api config to the last version with a numeric tax rate, then confirm order creation works.

Prevent a repeat

- Type-check business settings (tax rate must be a number between 0 and 1) in the config UI and in CI.
- Monitor each user journey separately: here only order creation failed while health checks looked fine.
- Require review for finance-owned config changes that reach production.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

HELD-OUT

ho-04-memory-leak

SYSTEM RECOVERED

100 / 100

recovery 60 · diagnosis 30 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 100%, p95 72ms over the last few seconds.
  - all probes passing but p95 latency is 72ms
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`memory_leak` on `api`: Feature flag enabling an unbounded in-process cache (OOM loop).
Root causes to name: `api/bad_config_deploy or api/memory_exhaustion`

MEND AI'S INCIDENT REPORT

`api/bad_config_deploy`: api is crash-looping: state=running restarts=2; api was deployed 0.7 min ago: v2 'enable local response cache to reduce redis load' by ci-bot

Summary: [confirmed] roll back the api config to v1 (undoing v2) (confidence 0.51; evidence: api is crash-looping: state=running restarts=2). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.

INVESTIGATION AND ACTIONS · 29 TOOL CALLS · 1 STATE CHANGES · 33.3 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.1s	OBSERVE	`check_endpoints()`
3	0.1s	OBSERVE	`get_recent_changes(limit=20)`
4	0.1s	OBSERVE	`get_metrics()`
5	2.5s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`
6	2.5s	OBSERVE	`get_logs(service=api, since_seconds=120, tail=400)`
7	2.5s	OBSERVE	`get_logs(service=worker, since_seconds=120, tail=400)`
8	2.6s	OBSERVE	`get_logs(service=postgres, since_seconds=120, tail=400)`
9	2.6s	OBSERVE	`get_logs(service=redis, since_seconds=120, tail=400)`
10	2.6s	OBSERVE	`get_config(service=api)`
11	2.6s	OBSERVE	`get_config(service=api, version=2, diff_against=1)`
12	2.6s	OBSERVE	`run_diagnostic(service=api, check=disk)`
13	2.7s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
14	2.8s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
15	2.9s	ACT	`rollback_config(service=api, to_version=1)`
16	21.5s	OBSERVE	`list_services()`
17	21.6s	OBSERVE	`check_endpoints()`

#	TIME	TYPE	TOOL CALL
18	21.7s	OBSERVE	`get_recent_changes(limit=20)`
19	21.7s	OBSERVE	`get_metrics()`
20	24.0s	OBSERVE	`get_logs(service=gateway, since_seconds=9, tail=400)`
21	24.0s	OBSERVE	`get_logs(service=api, since_seconds=9, tail=400)`
22	24.1s	OBSERVE	`get_logs(service=worker, since_seconds=9, tail=400)`
23	24.1s	OBSERVE	`get_logs(service=postgres, since_seconds=9, tail=400)`
24	24.1s	OBSERVE	`get_logs(service=redis, since_seconds=9, tail=400)`
25	24.1s	OBSERVE	`run_diagnostic(service=api, check=disk)`
26	24.2s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
27	24.3s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
28	24.4s	OBSERVE	`verify_recovery()`
29	33.3s	FINAL	`submit_report(root_causes=[{'component': 'api', 'category': 'bad_config_deploy', 'evidence': "api is crash-looping: state=running restarts=2; api was deployed 0.7 min ago: v2 'enable local response cache to reduce redis load' by ci-bot"}], recovered=True, summary=[confirmed] roll back the api config to v1 (undoing v2) (confidence 0.51; evidence: api is crash-looping: state=running restarts=2). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.)`

INDEPENDENT VERIFICATION

100% user traffic success	**26 ms** p95 latency	**0** collateral actions	**1.00** diagnosis F1

RECOMMENDATIONS

Immediate fix **DONE BY THE AGENT**

- Roll back the config change that enabled the unbounded local cache, then confirm memory stays flat and OOM kills stop.

Prevent a repeat

- Give every in-process cache a size limit and eviction policy.
- Roll out feature flags gradually, watching memory, before enabling them everywhere.
- Alert on OOM kills and on memory growth trends, not only on outages.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

HELD-OUT

ho-04-memory-leak

SYSTEM RECOVERED

100 / 100

recovery 60 · diagnosis 30 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 100%, p95 53ms over the last few seconds.
  - all probes passing but p95 latency is 53ms
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`memory_leak` on `api`: Feature flag enabling an unbounded in-process cache (OOM loop).
Root causes to name: `api/bad_config_deploy or api/memory_exhaustion`

MEND AI'S INCIDENT REPORT

`api/bad_config_deploy`: api is crash-looping: state=running restarts=2; api was deployed 0.7 min ago: v2 'enable local response cache to reduce redis load' by ci-bot

Summary: [confirmed] roll back the api config to v1 (undoing v2) (confidence 0.51; evidence: api is crash-looping: state=running restarts=2). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.

INVESTIGATION AND ACTIONS · 29 TOOL CALLS · 1 STATE CHANGES · 33.3 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.1s	OBSERVE	`check_endpoints()`
3	0.1s	OBSERVE	`get_recent_changes(limit=20)`
4	0.1s	OBSERVE	`get_metrics()`
5	2.4s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`
6	2.5s	OBSERVE	`get_logs(service=api, since_seconds=120, tail=400)`
7	2.5s	OBSERVE	`get_logs(service=worker, since_seconds=120, tail=400)`
8	2.5s	OBSERVE	`get_logs(service=postgres, since_seconds=120, tail=400)`
9	2.5s	OBSERVE	`get_logs(service=redis, since_seconds=120, tail=400)`
10	2.5s	OBSERVE	`get_config(service=api)`
11	2.5s	OBSERVE	`get_config(service=api, version=2, diff_against=1)`
12	2.5s	OBSERVE	`run_diagnostic(service=api, check=disk)`
13	2.6s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
14	2.8s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
15	2.8s	ACT	`rollback_config(service=api, to_version=1)`
16	21.4s	OBSERVE	`list_services()`
17	21.6s	OBSERVE	`check_endpoints()`

#	TIME	TYPE	TOOL CALL
18	21.6s	OBSERVE	`get_recent_changes(limit=20)`
19	21.6s	OBSERVE	`get_metrics()`
20	23.9s	OBSERVE	`get_logs(service=gateway, since_seconds=9, tail=400)`
21	24.0s	OBSERVE	`get_logs(service=api, since_seconds=9, tail=400)`
22	24.0s	OBSERVE	`get_logs(service=worker, since_seconds=9, tail=400)`
23	24.0s	OBSERVE	`get_logs(service=postgres, since_seconds=9, tail=400)`
24	24.1s	OBSERVE	`get_logs(service=redis, since_seconds=9, tail=400)`
25	24.1s	OBSERVE	`run_diagnostic(service=api, check=disk)`
26	24.2s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
27	24.3s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
28	24.4s	OBSERVE	`verify_recovery()`
29	33.3s	FINAL	`submit_report(root_causes=[{'component': 'api', 'category': 'bad_config_deploy', 'evidence': "api is crash-looping: state=running restarts=2; api was deployed 0.7 min ago: v2 'enable local response cache to reduce redis load' by ci-bot"}], recovered=True, summary=[confirmed] roll back the api config to v1 (undoing v2) (confidence 0.51; evidence: api is crash-looping: state=running restarts=2). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.)`

INDEPENDENT VERIFICATION

100% user traffic success	**23 ms** p95 latency	**0** collateral actions	**1.00** diagnosis F1

RECOMMENDATIONS

Immediate fix **DONE BY THE AGENT**

- Roll back the config change that enabled the unbounded local cache, then confirm memory stays flat and OOM kills stop.

Prevent a repeat

- Give every in-process cache a size limit and eviction policy.
- Roll out feature flags gradually, watching memory, before enabling them everywhere.
- Alert on OOM kills and on memory growth trends, not only on outages.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

HELD-OUT

ho-05-db-lock

SYSTEM RECOVERED

100 / 100

recovery 60 · diagnosis 30 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 60%, p95 2017ms over the last few seconds.
  - GET /api/orders/recent: 1/1 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 1/1 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`db_lock` on `postgres`: A stuck migration holds an exclusive lock on the orders table.
Root causes to name: `postgres/db_lock_contention`

MEND AI'S INCIDENT REPORT

`postgres/db_lock_contention`: 1 session(s) blocked on locks for up to 0s (run_diagnostic db_locks); root blocker pid 130 app=migration-runner state=active query=BEGIN; LOCK TABLE orders IN ACCESS EXCLUSIVE MODE; SELECT pg; 15 timeout errors in api/worker logs

Summary: [confirmed] terminate the session(s) holding the blocking lock: pids 130 (confidence 0.78; evidence: 1 session(s) blocked on locks for up to 0s (run_diagnostic db_locks)). Prevent: set lock_timeout for migrations and run them in maintenance windows.

INVESTIGATION AND ACTIONS · 27 TOOL CALLS · 1 STATE CHANGES · 23.7 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.1s	OBSERVE	`check_endpoints()`
3	4.2s	OBSERVE	`get_recent_changes(limit=20)`
4	4.2s	OBSERVE	`get_metrics()`
5	6.5s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`
6	6.5s	OBSERVE	`get_logs(service=api, since_seconds=120, tail=400)`
7	6.5s	OBSERVE	`get_logs(service=worker, since_seconds=120, tail=400)`
8	6.6s	OBSERVE	`get_logs(service=postgres, since_seconds=120, tail=400)`
9	6.6s	OBSERVE	`get_logs(service=redis, since_seconds=120, tail=400)`
10	6.6s	OBSERVE	`run_diagnostic(service=api, check=disk)`
11	6.7s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
12	6.8s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
13	6.9s	ACT	`terminate_db_sessions(pid=130)`
14	12.0s	OBSERVE	`list_services()`
15	12.1s	OBSERVE	`check_endpoints()`

#	TIME	TYPE	TOOL CALL
16	12.3s	OBSERVE	`get_recent_changes(limit=20)`
17	12.3s	OBSERVE	`get_metrics()`
18	14.6s	OBSERVE	`get_logs(service=gateway, since_seconds=8, tail=400)`
19	14.6s	OBSERVE	`get_logs(service=api, since_seconds=8, tail=400)`
20	14.7s	OBSERVE	`get_logs(service=worker, since_seconds=8, tail=400)`
21	14.7s	OBSERVE	`get_logs(service=postgres, since_seconds=8, tail=400)`
22	14.7s	OBSERVE	`get_logs(service=redis, since_seconds=8, tail=400)`
23	14.7s	OBSERVE	`run_diagnostic(service=api, check=disk)`
24	14.8s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
25	14.9s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
26	15.0s	OBSERVE	`verify_recovery()`
27	23.8s	FINAL	`submit_report(root_causes=[{'component': 'postgres', 'category': 'db_lock_contention', 'evidence': '1 session(s) blocked on locks for up to 0s (run_diagnostic db_locks); root blocker pid 130 app=migration-runner state=active query=BEGIN; LOCK TABLE orders IN ACCESS EXCLUSIVE MODE; SELECT pg; 15 timeout errors in api/worker logs'}], recovered=True, summary=[confirmed] terminate the session(s) holding the blocking lock: pids 130 (confidence 0.78; evidence: 1 session(s) blocked on locks for up to 0s (run_diagnostic db_locks)). Prevent: set lock_timeout for migrations and run them in maintenance windows.)`

INDEPENDENT VERIFICATION

100% user traffic success	**24 ms** p95 latency	**0** collateral actions	**1.00** diagnosis F1

RECOMMENDATIONS

Immediate fix **DONE BY THE AGENT**

- Terminate the migration-runner session holding the exclusive lock on orders (terminate_db_sessions).

Prevent a repeat

- Set lock_timeout and statement_timeout for migrations so they fail instead of blocking traffic.
- Run schema migrations with online, non-blocking patterns during low-traffic windows.
- Alert on sessions blocked by locks for more than a few seconds.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

HELD-OUT

ho-05-db-lock

SYSTEM RECOVERED

100 / 100

recovery 60 · diagnosis 30 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 60%, p95 2020ms over the last few seconds.
  - GET /api/orders/recent: 1/1 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 1/1 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`db_lock` on `postgres`: A stuck migration holds an exclusive lock on the orders table.
Root causes to name: `postgres/db_lock_contention`

MEND AI'S INCIDENT REPORT

`postgres/db_lock_contention`: 1 session(s) blocked on locks for up to 1s (run_diagnostic db_locks); root blocker pid 136 app=migration-runner state=active query=BEGIN; LOCK TABLE orders IN ACCESS EXCLUSIVE MODE; SELECT pg; 15 timeout errors in api/worker logs

Summary: [confirmed] terminate the session(s) holding the blocking lock: pids 136 (confidence 0.78; evidence: 1 session(s) blocked on locks for up to 1s (run_diagnostic db_locks)). Prevent: set lock_timeout for migrations and run them in maintenance windows.

INVESTIGATION AND ACTIONS · 27 TOOL CALLS · 1 STATE CHANGES · 23.6 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.1s	OBSERVE	`check_endpoints()`
3	4.2s	OBSERVE	`get_recent_changes(limit=20)`
4	4.2s	OBSERVE	`get_metrics()`
5	6.5s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`
6	6.5s	OBSERVE	`get_logs(service=api, since_seconds=120, tail=400)`
7	6.5s	OBSERVE	`get_logs(service=worker, since_seconds=120, tail=400)`
8	6.5s	OBSERVE	`get_logs(service=postgres, since_seconds=120, tail=400)`
9	6.6s	OBSERVE	`get_logs(service=redis, since_seconds=120, tail=400)`
10	6.6s	OBSERVE	`run_diagnostic(service=api, check=disk)`
11	6.7s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
12	6.8s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
13	6.9s	ACT	`terminate_db_sessions(pid=136)`
14	12.0s	OBSERVE	`list_services()`
15	12.1s	OBSERVE	`check_endpoints()`

#	TIME	TYPE	TOOL CALL
16	12.2s	OBSERVE	`get_recent_changes(limit=20)`
17	12.2s	OBSERVE	`get_metrics()`
18	14.5s	OBSERVE	`get_logs(service=gateway, since_seconds=8, tail=400)`
19	14.5s	OBSERVE	`get_logs(service=api, since_seconds=8, tail=400)`
20	14.5s	OBSERVE	`get_logs(service=worker, since_seconds=8, tail=400)`
21	14.6s	OBSERVE	`get_logs(service=postgres, since_seconds=8, tail=400)`
22	14.6s	OBSERVE	`get_logs(service=redis, since_seconds=8, tail=400)`
23	14.6s	OBSERVE	`run_diagnostic(service=api, check=disk)`
24	14.7s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
25	14.8s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
26	14.9s	OBSERVE	`verify_recovery()`
27	23.6s	FINAL	`submit_report(root_causes=[{'component': 'postgres', 'category': 'db_lock_contention', 'evidence': '1 session(s) blocked on locks for up to 1s (run_diagnostic db_locks); root blocker pid 136 app=migration-runner state=active query=BEGIN; LOCK TABLE orders IN ACCESS EXCLUSIVE MODE; SELECT pg; 15 timeout errors in api/worker logs'}], recovered=True, summary=[confirmed] terminate the session(s) holding the blocking lock: pids 136 (confidence 0.78; evidence: 1 session(s) blocked on locks for up to 1s (run_diagnostic db_locks)). Prevent: set lock_timeout for migrations and run them in maintenance windows.)`

INDEPENDENT VERIFICATION

100% user traffic success	**22 ms** p95 latency	**0** collateral actions	**1.00** diagnosis F1

RECOMMENDATIONS

Immediate fix **DONE BY THE AGENT**

- Terminate the migration-runner session holding the exclusive lock on orders (terminate_db_sessions).

Prevent a repeat

- Set lock_timeout and statement_timeout for migrations so they fail instead of blocking traffic.
- Run schema migrations with online, non-blocking patterns during low-traffic windows.
- Alert on sessions blocked by locks for more than a few seconds.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

HELD-OUT

ho-06-conn-exhaustion

SYSTEM RECOVERED

100 / 100

recovery 60 · diagnosis 30 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 40%, p95 16ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: connection
failed: connection to serv)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`conn_exhaustion` on `worker`: A backfill job in the worker leaks DB connections (and reconnects). Root causes to name: `worker/db_connection_exhaustion` or `postgres/db_connection_exhaustion`

MEND AI'S INCIDENT REPORT

`worker/db_connection_exhaustion`: postgres has 38/40 client connections (get_metrics); 44 'too many clients' errors, e.g. 2026-10-30 03:31:21.814 UTC [316] FATAL: remaining connection slots are reserved for roles with the SUPERUSER attribute; application 'report-backfill' holds 37 sessions (run_diagnostic db_activity)

Summary: [confirmed] stop 'python /app/tools/report_backfill.py' on worker (it keeps reconnecting), then terminate the 'report-backfill' database sessions to free connection slots (confidence 0.85; evidence: postgres has 38/40 client connections (get_metrics)). Prevent: use a connection pool with a per-client cap and alert on connection saturation.

INVESTIGATION AND ACTIONS · 31 TOOL CALLS · 2 STATE CHANGES · 20.2 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.1s	OBSERVE	`check_endpoints()`
3	0.2s	OBSERVE	`get_recent_changes(limit=20)`
4	0.2s	OBSERVE	`get_metrics()`
5	2.5s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`
6	2.5s	OBSERVE	`get_logs(service=api, since_seconds=120, tail=400)`
7	2.5s	OBSERVE	`get_logs(service=worker, since_seconds=120, tail=400)`
8	2.5s	OBSERVE	`get_logs(service=postgres, since_seconds=120, tail=400)`
9	2.6s	OBSERVE	`get_logs(service=redis, since_seconds=120, tail=400)`
10	2.6s	OBSERVE	`run_diagnostic(service=api, check=disk)`
11	2.7s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
12	2.8s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`

#	TIME	TYPE	TOOL CALL
13	2.9s	OBSERVE	`run_diagnostic(service=postgres, check=db_activity)`
14	3.1s	OBSERVE	`run_diagnostic(service=worker, check=processes)`
15	3.3s	ACT	`kill_process(service=worker, pid=6)`
16	3.4s	ACT	`terminate_db_sessions(application_name=report-backfill)`
17	8.4s	OBSERVE	`list_services()`
18	8.6s	OBSERVE	`check_endpoints()`
19	8.6s	OBSERVE	`get_recent_changes(limit=20)`
20	8.6s	OBSERVE	`get_metrics()`
21	10.9s	OBSERVE	`get_logs(service=gateway, since_seconds=8, tail=400)`
22	10.9s	OBSERVE	`get_logs(service=api, since_seconds=8, tail=400)`
23	11.0s	OBSERVE	`get_logs(service=worker, since_seconds=8, tail=400)`
24	11.0s	OBSERVE	`get_logs(service=postgres, since_seconds=8, tail=400)`
25	11.0s	OBSERVE	`get_logs(service=redis, since_seconds=8, tail=400)`
26	11.1s	OBSERVE	`run_diagnostic(service=api, check=disk)`
27	11.2s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
28	11.2s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
29	11.3s	OBSERVE	`run_diagnostic(service=postgres, check=db_activity)`
30	11.5s	OBSERVE	`verify_recovery()`
31	20.2s	FINAL	`submit_report(root_causes=[{'component': 'worker', 'category': 'db_connection_exhaustion', 'evidence': "postgres has 38/40 client connections (get_metrics); 44 'too many clients' errors, e.g. 2026-10-30 03:31:21.814 UTC [316] FATAL: remaining connection slots are reserved for roles with the SUPERUSER attribute; application 'report-backfill' holds 37 sessions (run_diagnostic db_activity)"}], recovered=True, summary=[confirmed] stop 'python /app/tools/report_backfill.py' on worker (it keeps reconnecting), then terminate the 'report-backfill' database sessions to free connection slots (confidence 0.85; evidence: postgres has 38/40 client connections (get_metrics)). Prevent: use a connection pool with a per-client cap and alert on connection saturation.)`

INDEPENDENT VERIFICATION

100% user traffic success	**23 ms** p95 latency	**0** collateral actions	**1.00** diagnosis F1

RECOMMENDATIONS

Immediate fix **DONE BY THE AGENT**

- Stop the leaking report_backfill job in the worker and terminate its database sessions.

Prevent a repeat

- Use a bounded connection pool in every job, and close connections in finally blocks.
- Give each application its own connection limit (per-role CONNECTION LIMIT or a pooler such as PgBouncer).
- Alert when database connections pass 80% of max_connections.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

HELD-OUT

ho-06-conn-exhaustion

SYSTEM RECOVERED

100 / 100

recovery 60 · diagnosis 30 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 40%, p95 16ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: connection
failed: connection to serv)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`conn_exhaustion` on `worker`: A backfill job in the worker leaks DB connections (and reconnects). Root causes to name: `worker/db_connection_exhaustion` or `postgres/db_connection_exhaustion`

MEND AI'S INCIDENT REPORT

`worker/db_connection_exhaustion`: postgres has 38/40 client connections (get_metrics); 44 'too many clients' errors, e.g. 2026-10-30 03:53:18.521 UTC [329] FATAL: remaining connection slots are reserved for roles with the SUPERUSER attribute; application 'report-backfill' holds 37 sessions (run_diagnostic db_activity)

Summary: [confirmed] stop 'python /app/tools/report_backfill.py' on worker (it keeps reconnecting), then terminate the 'report-backfill' database sessions to free connection slots (confidence 0.85; evidence: postgres has 38/40 client connections (get_metrics)). Prevent: use a connection pool with a per-client cap and alert on connection saturation.

INVESTIGATION AND ACTIONS · 31 TOOL CALLS · 2 STATE CHANGES · 20.1 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.1s	OBSERVE	`check_endpoints()`
3	0.2s	OBSERVE	`get_recent_changes(limit=20)`
4	0.2s	OBSERVE	`get_metrics()`
5	2.4s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`
6	2.5s	OBSERVE	`get_logs(service=api, since_seconds=120, tail=400)`
7	2.5s	OBSERVE	`get_logs(service=worker, since_seconds=120, tail=400)`
8	2.5s	OBSERVE	`get_logs(service=postgres, since_seconds=120, tail=400)`
9	2.5s	OBSERVE	`get_logs(service=redis, since_seconds=120, tail=400)`
10	2.6s	OBSERVE	`run_diagnostic(service=api, check=disk)`
11	2.6s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
12	2.7s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`

#	TIME	TYPE	TOOL CALL
13	2.8s	OBSERVE	`run_diagnostic(service=postgres, check=db_activity)`
14	3.0s	OBSERVE	`run_diagnostic(service=worker, check=processes)`
15	3.2s	ACT	`kill_process(service=worker, pid=7)`
16	3.3s	ACT	`terminate_db_sessions(application_name=report-backfill)`
17	8.4s	OBSERVE	`list_services()`
18	8.5s	OBSERVE	`check_endpoints()`
19	8.6s	OBSERVE	`get_recent_changes(limit=20)`
20	8.6s	OBSERVE	`get_metrics()`
21	10.8s	OBSERVE	`get_logs(service=gateway, since_seconds=8, tail=400)`
22	10.9s	OBSERVE	`get_logs(service=api, since_seconds=8, tail=400)`
23	10.9s	OBSERVE	`get_logs(service=worker, since_seconds=8, tail=400)`
24	10.9s	OBSERVE	`get_logs(service=postgres, since_seconds=8, tail=400)`
25	11.0s	OBSERVE	`get_logs(service=redis, since_seconds=8, tail=400)`
26	11.0s	OBSERVE	`run_diagnostic(service=api, check=disk)`
27	11.1s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
28	11.1s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
29	11.2s	OBSERVE	`run_diagnostic(service=postgres, check=db_activity)`
30	11.4s	OBSERVE	`verify_recovery()`
31	20.1s	FINAL	`submit_report(root_causes=[{'component': 'worker', 'category': 'db_connection_exhaustion', 'evidence': "postgres has 38/40 client connections (get_metrics); 44 'too many clients' errors, e.g. 2026-10-30 03:53:18.521 UTC [329] FATAL: remaining connection slots are reserved for roles with the SUPERUSER attribute; application 'report-backfill' holds 37 sessions (run_diagnostic db_activity)"}], recovered=True, summary=[confirmed] stop 'python /app/tools/report_backfill.py' on worker (it keeps reconnecting), then terminate the 'report-backfill' database sessions to free connection slots (confidence 0.85; evidence: postgres has 38/40 client connections (get_metrics)). Prevent: use a connection pool with a per-client cap and alert on connection saturation.)`

INDEPENDENT VERIFICATION

100% user traffic success	**22 ms** p95 latency	**0** collateral actions	**1.00** diagnosis F1

RECOMMENDATIONS

Immediate fix **DONE BY THE AGENT**

- Stop the leaking report_backfill job in the worker and terminate its database sessions.

Prevent a repeat

- Use a bounded connection pool in every job, and close connections in finally blocks.
- Give each application its own connection limit (per-role CONNECTION LIMIT or a pooler such as PgBouncer).
- Alert when database connections pass 80% of max_connections.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

HELD-OUT

ho-07-redis-partition

SYSTEM RECOVERED

100 / 100

recovery 60 · diagnosis 30 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 80%, p95 23ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [200] ({"checks":{"cache":"error: Error -2 connecting to
redis:6379. Name does not reso)
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`network_partition` on `redis`: A backend service is cut off from the backend network.
Root causes to name: `redis/network_partition`

MEND AI'S INCIDENT REPORT

`redis/network_partition`: redis is not attached to the backend network it belongs on (list_services); 44 name-resolution errors in logs, e.g. probe /api/health: {"checks":{"cache":"error: Error -2 connecting to redis:6379. Name does not resolve.","database":"ok

Summary: [confirmed] re-attach redis to the backend network (restores its DNS name) (confidence 0.94; evidence: redis is not attached to the backend network it belongs on (list_services)). Prevent: monitor container network attachment and DNS resolution between services.

INVESTIGATION AND ACTIONS · 27 TOOL CALLS · 1 STATE CHANGES · 22.7 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.1s	OBSERVE	`check_endpoints()`
3	0.2s	OBSERVE	`get_recent_changes(limit=20)`
4	0.2s	OBSERVE	`get_metrics()`
5	2.5s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`
6	2.5s	OBSERVE	`get_logs(service=api, since_seconds=120, tail=400)`
7	2.5s	OBSERVE	`get_logs(service=worker, since_seconds=120, tail=400)`
8	2.6s	OBSERVE	`get_logs(service=postgres, since_seconds=120, tail=400)`
9	2.6s	OBSERVE	`get_logs(service=redis, since_seconds=120, tail=400)`
10	2.6s	OBSERVE	`run_diagnostic(service=api, check=disk)`
11	2.7s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
12	2.8s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
13	2.9s	ACT	`reconnect_network(service=redis, network=backend)`
14	11.0s	OBSERVE	`list_services()`
15	11.1s	OBSERVE	`check_endpoints()`
16	11.2s	OBSERVE	`get_recent_changes(limit=20)`

MEND

#	TIME	TYPE	TOOL CALL
17	11.2s	OBSERVE	`get_metrics()`
18	13.5s	OBSERVE	`get_logs(service=gateway, since_seconds=9, tail=400)`
19	13.5s	OBSERVE	`get_logs(service=api, since_seconds=9, tail=400)`
20	13.5s	OBSERVE	`get_logs(service=worker, since_seconds=9, tail=400)`
21	13.6s	OBSERVE	`get_logs(service=postgres, since_seconds=9, tail=400)`
22	13.6s	OBSERVE	`get_logs(service=redis, since_seconds=9, tail=400)`
23	13.6s	OBSERVE	`run_diagnostic(service=api, check=disk)`
24	13.7s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
25	13.8s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
26	13.9s	OBSERVE	`verify_recovery()`
27	22.7s	FINAL	`submit_report(root_causes=[{'component': 'redis', 'category': 'network_partition', 'evidence': 'redis is not attached to the backend network it belongs on (list_services); 44 name-resolution errors in logs, e.g. probe /api/health: {"checks":{"cache":"error: Error -2 connecting to redis:6379. Name does not resolve.","database":"ok'}], recovered=True, summary=[confirmed] re-attach redis to the backend network (restores its DNS name) (confidence 0.94; evidence: redis is not attached to the backend network it belongs on (list_services)). Prevent: monitor container network attachment and DNS resolution between services.)`

INDEPENDENT VERIFICATION

100% user traffic success	**22 ms** p95 latency	**0** collateral actions	**1.00** diagnosis F1

RECOMMENDATIONS

Immediate fix **DONE BY THE AGENT**

- Reconnect redis to the backend network (reconnect_network), then confirm its dependents recover.

Prevent a repeat

- Manage network attachments only through docker compose or infrastructure code, never by hand.
- Add connectivity checks between dependent services to monitoring.
- Alert on dependency health (database/cache unreachable) with the exact component named.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

HELD-OUT

ho-07-redis-partition

SYSTEM RECOVERED

100 / 100

recovery 60 · diagnosis 30 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 80%, p95 26ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [200] ({"checks":{"cache":"error: Error -2 connecting to
redis:6379. Name does not reso)
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`network_partition` on `redis`: A backend service is cut off from the backend network.
Root causes to name: `redis/network_partition`

MEND AI'S INCIDENT REPORT

`redis/network_partition`: redis is not attached to the backend network it belongs on (list_services); 44 name-resolution errors in logs, e.g. probe /api/health: {"checks":{"cache":"error: Error -2 connecting to redis:6379. Name does not resolve.","database":"ok

Summary: [confirmed] re-attach redis to the backend network (restores its DNS name) (confidence 0.94; evidence: redis is not attached to the backend network it belongs on (list_services)). Prevent: monitor container network attachment and DNS resolution between services.

INVESTIGATION AND ACTIONS · 27 TOOL CALLS · 1 STATE CHANGES · 22.6 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.1s	OBSERVE	`check_endpoints()`
3	0.2s	OBSERVE	`get_recent_changes(limit=20)`
4	0.2s	OBSERVE	`get_metrics()`
5	2.5s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`
6	2.5s	OBSERVE	`get_logs(service=api, since_seconds=120, tail=400)`
7	2.6s	OBSERVE	`get_logs(service=worker, since_seconds=120, tail=400)`
8	2.6s	OBSERVE	`get_logs(service=postgres, since_seconds=120, tail=400)`
9	2.6s	OBSERVE	`get_logs(service=redis, since_seconds=120, tail=400)`
10	2.6s	OBSERVE	`run_diagnostic(service=api, check=disk)`
11	2.7s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
12	2.8s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
13	2.9s	ACT	`reconnect_network(service=redis, network=backend)`
14	11.0s	OBSERVE	`list_services()`
15	11.1s	OBSERVE	`check_endpoints()`
16	11.2s	OBSERVE	`get_recent_changes(limit=20)`

#	TIME	TYPE	TOOL CALL
17	11.2s	OBSERVE	`get_metrics()`
18	13.5s	OBSERVE	`get_logs(service=gateway, since_seconds=9, tail=400)`
19	13.5s	OBSERVE	`get_logs(service=api, since_seconds=9, tail=400)`
20	13.5s	OBSERVE	`get_logs(service=worker, since_seconds=9, tail=400)`
21	13.5s	OBSERVE	`get_logs(service=postgres, since_seconds=9, tail=400)`
22	13.6s	OBSERVE	`get_logs(service=redis, since_seconds=9, tail=400)`
23	13.6s	OBSERVE	`run_diagnostic(service=api, check=disk)`
24	13.7s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
25	13.8s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
26	13.9s	OBSERVE	`verify_recovery()`
27	22.6s	FINAL	`submit_report(root_causes=[{'component': 'redis', 'category': 'network_partition', 'evidence': 'redis is not attached to the backend network it belongs on (list_services); 44 name-resolution errors in logs, e.g. probe /api/health: {"checks":{"cache":"error: Error -2 connecting to redis:6379. Name does not resolve.","database":"ok'}], recovered=True, summary=[confirmed] re-attach redis to the backend network (restores its DNS name) (confidence 0.94; evidence: redis is not attached to the backend network it belongs on (list_services)). Prevent: monitor container network attachment and DNS resolution between services.)`

INDEPENDENT VERIFICATION

100% user traffic success	**24 ms** p95 latency	**0** collateral actions	**1.00** diagnosis F1

RECOMMENDATIONS

Immediate fix **DONE BY THE AGENT**

- Reconnect redis to the backend network (reconnect_network), then confirm its dependents recover.

Prevent a repeat

- Manage network attachments only through docker compose or infrastructure code, never by hand.
- Add connectivity checks between dependent services to monitoring.
- Alert on dependency health (database/cache unreachable) with the exact component named.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

HELD-OUT

ho-08-rollback-trap

SYSTEM RECOVERED

100 / 100

recovery 60 · diagnosis 30 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 20%, p95 17ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: connection
failed: connection to serv)
  - GET /api/products: 5/5 failed, statuses [503] ({"error":"database unavailable"}
)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`bad_db_password` on `api`: Config deploy with wrong DB password.

Red herrings planted: `benign_api_deploy`

Root causes to name: `api/bad_config_deploy`

MEND AI'S INCIDENT REPORT

`api/bad_config_deploy`: 19 log line(s) on api: the database rejected the client's credentials, e.g. 2026-10-30T03:34:39 ERROR api db query failed (products): connection failed: connection to server at "172.18.0.3", port 5432 failed: FATAL: ; v2 changed password (get_config diff), matching the errors; api was deployed 0.4 min ago: v3 'tune cache TTL' by dev-ahmed

Summary: [confirmed] roll back the api config to v1 (undoing v2) (confidence 0.76; evidence: 19 log line(s) on api: the database rejected the client's credentials, e.g. 2026-10-30T03:34:39 ERROR api db query failed (products): connection failed: connection to server at "172.18.0.3", port 5432 failed: FATAL:). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.

INVESTIGATION AND ACTIONS · 30 TOOL CALLS · 1 STATE CHANGES · 33.2 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.1s	OBSERVE	`check_endpoints()`
3	0.2s	OBSERVE	`get_recent_changes(limit=20)`
4	0.2s	OBSERVE	`get_metrics()`
5	2.5s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`
6	2.6s	OBSERVE	`get_logs(service=api, since_seconds=120, tail=400)`
7	2.6s	OBSERVE	`get_logs(service=worker, since_seconds=120, tail=400)`
8	2.6s	OBSERVE	`get_logs(service=postgres, since_seconds=120, tail=400)`
9	2.7s	OBSERVE	`get_logs(service=redis, since_seconds=120, tail=400)`
10	2.7s	OBSERVE	`get_config(service=api)`
11	2.7s	OBSERVE	`get_config(service=api, version=3, diff_against=2)`

#	TIME	TYPE	TOOL CALL
12	2.7s	OBSERVE	`get_config(service=api, version=2, diff_against=1)`
13	2.7s	OBSERVE	`run_diagnostic(service=api, check=disk)`
14	2.8s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
15	2.9s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
16	3.0s	ACT	`rollback_config(service=api, to_version=1)`
17	21.6s	OBSERVE	`list_services()`
18	21.7s	OBSERVE	`check_endpoints()`
19	21.8s	OBSERVE	`get_recent_changes(limit=20)`
20	21.8s	OBSERVE	`get_metrics()`
21	24.1s	OBSERVE	`get_logs(service=gateway, since_seconds=9, tail=400)`
22	24.1s	OBSERVE	`get_logs(service=api, since_seconds=9, tail=400)`
23	24.1s	OBSERVE	`get_logs(service=worker, since_seconds=9, tail=400)`
24	24.1s	OBSERVE	`get_logs(service=postgres, since_seconds=9, tail=400)`
25	24.2s	OBSERVE	`get_logs(service=redis, since_seconds=9, tail=400)`
26	24.2s	OBSERVE	`run_diagnostic(service=api, check=disk)`
27	24.3s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
28	24.4s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
29	24.5s	OBSERVE	`verify_recovery()`
30	33.2s	FINAL	`submit_report(root_causes=[{'component': 'api', 'category': 'bad_config_deploy', 'evidence': '19 log line(s) on api: the database rejected the client\'s credentials, e.g. 2026-10-30T03:34:39 ERROR api db query failed (products): connection failed: connection to server at "172.18.0.3", port 5432 failed: FATAL: ; v2 changed password (get_config diff), matching the errors; api was deployed 0.4 min ago: v3 \'tune cache TTL\' by dev-ahmed'}], recovered=True, summary=[confirmed] roll back the api config to v1 (undoing v2) (confidence 0.76; evidence: 19 log line(s) on api: the database rejected the client's credentials, e.g. 2026-10-30T03:34:39 ERROR api db query failed (products): connection failed: connection to server at "172.18.0.3", port 5432 failed: FATAL:). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.)`

INDEPENDENT VERIFICATION

100% user traffic success	**31 ms** p95 latency	**0** collateral actions	**1.00** diagnosis F1

RECOMMENDATIONS

Immediate fix **DONE BY THE AGENT**

- Roll back the api config to the last version with the working database password.

Prevent a repeat

- Rotate credentials through a secrets manager in step with the database, never by editing app config.
- Run a pre-deploy check that the new config can authenticate to the database.
- Deploy config as a canary and roll back automatically when health checks fail.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

HELD-OUT

ho-08-rollback-trap

SYSTEM RECOVERED

100 / 100

recovery 60 · diagnosis 30 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 20%, p95 19ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: connection
failed: connection to serv)
  - GET /api/products: 5/5 failed, statuses [503] ({"error":"database unavailable"}
)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`bad_db_password` on `api`: Config deploy with wrong DB password.

Red herrings planted: `benign_api_deploy`

Root causes to name: `api/bad_config_deploy`

MEND AI'S INCIDENT REPORT

`api/bad_config_deploy`: 19 log line(s) on api: the database rejected the client's credentials, e.g. 2026-10-30T03:56:35 ERROR api db query failed (products): connection failed: connection to server at "172.18.0.3", port 5432 failed: FATAL: ; v2 changed password (get_config diff), matching the errors; api was deployed 0.4 min ago: v3 'tune cache TTL' by dev-ahmed

Summary: [confirmed] roll back the api config to v1 (undoing v2) (confidence 0.76; evidence: 19 log line(s) on api: the database rejected the client's credentials, e.g. 2026-10-30T03:56:35 ERROR api db query failed (products): connection failed: connection to server at "172.18.0.3", port 5432 failed: FATAL:). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.

INVESTIGATION AND ACTIONS · 30 TOOL CALLS · 1 STATE CHANGES · 33.1 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.1s	OBSERVE	`check_endpoints()`
3	0.2s	OBSERVE	`get_recent_changes(limit=20)`
4	0.2s	OBSERVE	`get_metrics()`
5	2.5s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`
6	2.5s	OBSERVE	`get_logs(service=api, since_seconds=120, tail=400)`
7	2.5s	OBSERVE	`get_logs(service=worker, since_seconds=120, tail=400)`
8	2.5s	OBSERVE	`get_logs(service=postgres, since_seconds=120, tail=400)`
9	2.6s	OBSERVE	`get_logs(service=redis, since_seconds=120, tail=400)`
10	2.6s	OBSERVE	`get_config(service=api)`
11	2.6s	OBSERVE	`get_config(service=api, version=3, diff_against=2)`

#	TIME	TYPE	TOOL CALL
12	2.6s	OBSERVE	`get_config(service=api, version=2, diff_against=1)`
13	2.6s	OBSERVE	`run_diagnostic(service=api, check=disk)`
14	2.7s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
15	2.8s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
16	2.9s	ACT	`rollback_config(service=api, to_version=1)`
17	21.5s	OBSERVE	`list_services()`
18	21.6s	OBSERVE	`check_endpoints()`
19	21.7s	OBSERVE	`get_recent_changes(limit=20)`
20	21.7s	OBSERVE	`get_metrics()`
21	23.9s	OBSERVE	`get_logs(service=gateway, since_seconds=9, tail=400)`
22	23.9s	OBSERVE	`get_logs(service=api, since_seconds=9, tail=400)`
23	24.0s	OBSERVE	`get_logs(service=worker, since_seconds=9, tail=400)`
24	24.0s	OBSERVE	`get_logs(service=postgres, since_seconds=9, tail=400)`
25	24.0s	OBSERVE	`get_logs(service=redis, since_seconds=9, tail=400)`
26	24.1s	OBSERVE	`run_diagnostic(service=api, check=disk)`
27	24.2s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
28	24.3s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
29	24.4s	OBSERVE	`verify_recovery()`
30	33.1s	FINAL	`submit_report(root_causes=[{'component': 'api', 'category': 'bad_config_deploy', 'evidence': '19 log line(s) on api: the database rejected the client\'s credentials, e.g. 2026-10-30T03:56:35 ERROR api db query failed (products): connection failed: connection to server at "172.18.0.3", port 5432 failed: FATAL: ; v2 changed password (get_config diff), matching the errors; api was deployed 0.4 min ago: v3 \'tune cache TTL\' by dev-ahmed'}], recovered=True, summary=[confirmed] roll back the api config to v1 (undoing v2) (confidence 0.76; evidence: 19 log line(s) on api: the database rejected the client's credentials, e.g. 2026-10-30T03:56:35 ERROR api db query failed (products): connection failed: connection to server at "172.18.0.3", port 5432 failed: FATAL:). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.)`

INDEPENDENT VERIFICATION

100% user traffic success	**22 ms** p95 latency	**0** collateral actions	**1.00** diagnosis F1

RECOMMENDATIONS

Immediate fix **DONE BY THE AGENT**

- Roll back the api config to the last version with the working database password.

Prevent a repeat

- Rotate credentials through a secrets manager in step with the database, never by editing app config.
- Run a pre-deploy check that the new config can authenticate to the database.
- Deploy config as a canary and roll back automatically when health checks fail.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

HELD-OUT

ho-09-lock-behind-gateway

SYSTEM RECOVERED

100 / 100

recovery 60 · diagnosis 30 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 20%, p95 2ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [502] (<html>
<head><title>502 Bad Gateway</title></head>
<body>
<center><h1>502 Bad)
  - GET /api/products: 5/5 failed, statuses [502] (<html>
<head><title>502 Bad Gateway</title></head>
<body>
<center><h1>502 Bad)
  - GET /api/orders/recent: 5/5 failed, statuses [502] (<html>
<head><title>502 Bad Gateway</title></head>
<body>
<center><h1>502 Bad)
  - POST /api/orders: 5/5 failed, statuses [502] (<html>
<head><title>502 Bad Gateway</title></head>
<body>
<center><h1>502 Bad)
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`db_lock` on `postgres`: A stuck migration holds an exclusive lock on the orders table.
`gateway_bad_upstream` on `gateway`: Gateway config deploy with wrong upstream port.
Root causes to name: `postgres/db_lock_contention; gateway/bad_config_deploy`

MEND AI'S INCIDENT REPORT

`gateway/bad_config_deploy`: 12 log line(s) on gateway: the target accepted no connection on that port, e.g. 2026/10/30 03:36:20 [error] 27#27: *433 connect() failed (111: Connection refused) while connecting to upstream, client: 172.19.0.4, server:; api answers when called directly from the gateway, but proxied requests fail; v2 changed server (get_config diff), matching the errors
`postgres/db_lock_contention`: 1 session(s) blocked on locks for up to 1s (run_diagnostic db_locks); root blocker pid 134 app=migration-runner state=active query=BEGIN; LOCK TABLE orders IN ACCESS EXCLUSIVE MODE; SELECT pg; 7 timeout errors in api/worker logs
Summary: [confirmed] roll back the gateway config to v1 (undoing v2) (confidence 0.86; evidence: 12 log line(s) on gateway: the target accepted no connection on that port, e.g. 2026/10/30 03:36:20 [error] 27#27: *433 connect() failed (111: Connection refused) while connecting to upstream, client: 172.19.0.4, server:). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes. I [confirmed] terminate the session(s) holding the blocking lock: pids 134 (confidence 0.78; evidence: 1 session(s) blocked on locks for up to 1s (run_diagnostic db_locks)). Prevent: set lock_timeout for migrations and run them in maintenance windows.

INVESTIGATION AND ACTIONS · 43 TOOL CALLS · 2 STATE CHANGES · 34.9 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.1s	OBSERVE	`check_endpoints()`
3	0.1s	OBSERVE	`get_recent_changes(limit=20)`
4	0.1s	OBSERVE	`get_metrics()`

#	TIME	TYPE	TOOL CALL
5	2.5s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`
6	2.5s	OBSERVE	`get_logs(service=api, since_seconds=120, tail=400)`
7	2.5s	OBSERVE	`get_logs(service=worker, since_seconds=120, tail=400)`
8	2.5s	OBSERVE	`get_logs(service=postgres, since_seconds=120, tail=400)`
9	2.6s	OBSERVE	`get_logs(service=redis, since_seconds=120, tail=400)`
10	2.6s	OBSERVE	`run_diagnostic(service=gateway, check=upstream_http)`
11	2.7s	OBSERVE	`get_config(service=gateway)`
12	2.7s	OBSERVE	`get_config(service=gateway, version=2, diff_against=1)`
13	2.7s	OBSERVE	`run_diagnostic(service=api, check=disk)`
14	2.8s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
15	2.9s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
16	3.0s	ACT	`rollback_config(service=gateway, to_version=1)`
17	11.1s	OBSERVE	`list_services()`
18	11.3s	OBSERVE	`check_endpoints()`
19	15.3s	OBSERVE	`get_recent_changes(limit=20)`
20	15.3s	OBSERVE	`get_metrics()`
21	17.6s	OBSERVE	`get_logs(service=gateway, since_seconds=9, tail=400)`
22	17.7s	OBSERVE	`get_logs(service=api, since_seconds=9, tail=400)`
23	17.7s	OBSERVE	`get_logs(service=worker, since_seconds=9, tail=400)`
24	17.7s	OBSERVE	`get_logs(service=postgres, since_seconds=9, tail=400)`
25	17.8s	OBSERVE	`get_logs(service=redis, since_seconds=9, tail=400)`
26	17.8s	OBSERVE	`run_diagnostic(service=api, check=disk)`
27	17.9s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
28	18.0s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
29	18.1s	ACT	`terminate_db_sessions(pid=134)`
30	23.2s	OBSERVE	`list_services()`
31	23.3s	OBSERVE	`check_endpoints()`
32	23.4s	OBSERVE	`get_recent_changes(limit=20)`
33	23.4s	OBSERVE	`get_metrics()`
34	25.7s	OBSERVE	`get_logs(service=gateway, since_seconds=8, tail=400)`
35	25.7s	OBSERVE	`get_logs(service=api, since_seconds=8, tail=400)`
36	25.7s	OBSERVE	`get_logs(service=worker, since_seconds=8, tail=400)`
37	25.7s	OBSERVE	`get_logs(service=postgres, since_seconds=8, tail=400)`
38	25.8s	OBSERVE	`get_logs(service=redis, since_seconds=8, tail=400)`

MEND

#	TIME	TYPE	TOOL CALL
39	25.8s	OBSERVE	`run_diagnostic(service=api, check=disk)`
40	25.9s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
41	26.0s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
42	26.1s	OBSERVE	`verify_recovery()`
43	34.9s	FINAL	`submit_report(root_causes=[{'component': 'gateway', 'category': 'bad_config_deploy', 'evidence': '12 log line(s) on gateway: the target accepted no connection on that port, e.g. 2026/10/30 03:36:20 [error] 27#27: *433 connect() failed (111: Connection refused) while connecting to upstream, client: 172.19.0.4, server:; api answers when called directly from the gateway, but proxied requests fail; v2 changed server (get_config diff), matching the errors'}, {'component': 'postgres', 'category': 'db_lock_contention', 'evidence': '1 session(s) blocked on locks for up to 1s (run_diagnostic db_locks); root blocker pid 134 app=migration-runner state=active query=BEGIN; LOCK TABLE orders IN ACCESS EXCLUSIVE MODE; SELECT pg; 7 timeout errors in api/worker logs'}], recovered=True, summary=[confirmed] roll back the gateway config to v1 (undoing v2) (confidence 0.86; evidence: 12 log line(s) on gateway: the target accepted no connection on that port, e.g. 2026/10/30 03:36:20 [error] 27#27: *433 connect() failed (111: Connection refused) while connecting to upstream, client: 172.19.0.4, server:). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes. | [confirmed] terminate the session(s) holding the blocking lock: pids 134 (confidence 0.78; evidence: 1 session(s) blocked on locks for up to 1s (run_diagnostic db_locks)). Prevent: set lock_timeout for migrations and run them in maintenance windows.)`

INDEPENDENT VERIFICATION

100% user traffic success	**23 ms** p95 latency	**0** collateral actions	**1.00** diagnosis F1

RECOMMENDATIONS

Immediate fix **DONE BY THE AGENT**

- Terminate the migration-runner session holding the exclusive lock on orders (terminate_db_sessions).
- Roll back the gateway config to the last version with the correct api upstream port (api:8000).

Prevent a repeat

- Set lock_timeout and statement_timeout for migrations so they fail instead of blocking traffic.
- Run schema migrations with online, non-blocking patterns during low-traffic windows.
- Alert on sessions blocked by locks for more than a few seconds.
- Run nginx -t and an upstream connectivity check as part of every gateway deploy.
- Keep upstream addresses in one shared service registry instead of hand-edited configs.
- Deploy gateway changes with automatic rollback when 5xx rates jump.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

HELD-OUT

ho-09-lock-behind-gateway

STILL BROKEN

30 / 100

recovery 0 · diagnosis 20 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 20%, p95 1ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [502] (<html>
<head><title>502 Bad Gateway</title></head>
<body>
<center><h1>502 Bad)
  - GET /api/products: 5/5 failed, statuses [502] (<html>
<head><title>502 Bad Gateway</title></head>
<body>
<center><h1>502 Bad)
  - GET /api/orders/recent: 5/5 failed, statuses [502] (<html>
<head><title>502 Bad Gateway</title></head>
<body>
<center><h1>502 Bad)
  - POST /api/orders: 5/5 failed, statuses [502] (<html>
<head><title>502 Bad Gateway</title></head>
<body>
<center><h1>502 Bad)
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`db_lock` on `postgres`: A stuck migration holds an exclusive lock on the orders table.
`gateway_bad_upstream` on `gateway`: Gateway config deploy with wrong upstream port.
Root causes to name: `postgres/db_lock_contention; gateway/bad_config_deploy`

MEND AI'S INCIDENT REPORT

`gateway/bad_config_deploy`: 12 log line(s) on gateway: the target accepted no connection on that port, e.g. 2026/10/30 03:58:16 [error] 27#27: *435 connect() failed (111: Connection refused) while connecting to upstream, client: 172.19.0.4, server:; api answers when called directly from the gateway, but proxied requests fail; v2 changed server (get_config diff), matching the errors

Summary: [confirmed] roll back the gateway config to v1 (undoing v2) (confidence 0.86; evidence: 12 log line(s) on gateway: the target accepted no connection on that port, e.g. 2026/10/30 03:58:16 [error] 27#27: *435 connect() failed (111: Connection refused) while connecting to upstream, client: 172.19.0.4, server:). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.

INVESTIGATION AND ACTIONS · 30 TOOL CALLS · 1 STATE CHANGES · 28.3 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.1s	OBSERVE	`check_endpoints()`
3	0.1s	OBSERVE	`get_recent_changes(limit=20)`
4	0.1s	OBSERVE	`get_metrics()`
5	2.4s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`
6	2.5s	OBSERVE	`get_logs(service=api, since_seconds=120, tail=400)`
7	2.5s	OBSERVE	`get_logs(service=worker, since_seconds=120, tail=400)`

#	TIME	TYPE	TOOL CALL
8	2.5s	OBSERVE	`get_logs(service=postgres, since_seconds=120, tail=400)`
9	2.5s	OBSERVE	`get_logs(service=redis, since_seconds=120, tail=400)`
10	2.6s	OBSERVE	`run_diagnostic(service=gateway, check=upstream_http)`
11	2.7s	OBSERVE	`get_config(service=gateway)`
12	2.7s	OBSERVE	`get_config(service=gateway, version=2, diff_against=1)`
13	2.7s	OBSERVE	`run_diagnostic(service=api, check=disk)`
14	2.8s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
15	2.9s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
16	3.0s	ACT	`rollback_config(service=gateway, to_version=1)`
17	11.1s	OBSERVE	`list_services()`
18	11.2s	OBSERVE	`check_endpoints()`
19	15.3s	OBSERVE	`get_recent_changes(limit=20)`
20	15.3s	OBSERVE	`get_metrics()`
21	17.6s	OBSERVE	`get_logs(service=gateway, since_seconds=9, tail=400)`
22	17.6s	OBSERVE	`get_logs(service=api, since_seconds=9, tail=400)`
23	17.6s	OBSERVE	`get_logs(service=worker, since_seconds=9, tail=400)`
24	17.7s	OBSERVE	`get_logs(service=postgres, since_seconds=9, tail=400)`
25	17.7s	OBSERVE	`get_logs(service=redis, since_seconds=9, tail=400)`
26	17.7s	OBSERVE	`run_diagnostic(service=api, check=disk)`
27	17.8s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
28	17.9s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
29	18.0s	OBSERVE	`verify_recovery()`
30	28.4s	FINAL	`submit_report(root_causes=[{'component': 'gateway', 'category': 'bad_config_deploy', 'evidence': '12 log line(s) on gateway: the target accepted no connection on that port, e.g. 2026/10/30 03:58:16 [error] 27#27: *435 connect() failed (111: Connection refused) while connecting to upstream, client: 172.19.0.4, server:; api answers when called directly from the gateway, but proxied requests fail; v2 changed server (get_config diff), matching the errors'}], recovered=False, summary=[confirmed] roll back the gateway config to v1 (undoing v2) (confidence 0.86; evidence: 12 log line(s) on gateway: the target accepted no connection on that port, e.g. 2026/10/30 03:58:16 [error] 27#27: *435 connect() failed (111: Connection refused) while connecting to upstream, client: 172.19.0.4, server:). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.)`

INDEPENDENT VERIFICATION

60%	2021 ms	0	0.67
user traffic success	p95 latency	collateral actions	diagnosis F1

Findings: probe success 60% < 97%; probe p95 2021ms > 400ms; residual[db_lock:postgres]: migration-runner sessions: 1

RECOMMENDATIONS

Immediate fix **STILL NEEDED**

- Terminate the migration-runner session holding the exclusive lock on orders (terminate_db_sessions).
- Roll back the gateway config to the last version with the correct api upstream port (api:8000).

Prevent a repeat

- Set lock_timeout and statement_timeout for migrations so they fail instead of blocking traffic.
- Run schema migrations with online, non-blocking patterns during low-traffic windows.
- Alert on sessions blocked by locks for more than a few seconds.
- Run nginx -t and an upstream connectivity check as part of every gateway deploy.
- Keep upstream addresses in one shared service registry instead of hand-edited configs.
- Deploy gateway changes with automatic rollback when 5xx rates jump.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

HELD-OUT

ho-10-cpu-and-redis-partition

SYSTEM RECOVERED

100 / 100

recovery 60 · diagnosis 30 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 80%, p95 798ms over the last few seconds.
  - GET /api/health: 2/2 failed, statuses [200] ({"checks":{"cache":"error: Error -2 connecting to
redis:6379. Name does not reso)
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`cpu_hog` on `api`: A runaway search reindex job saturates the API's CPU.
`network_partition` on `redis`: A backend service is cut off from the backend network.
Red herrings planted: `benign_gateway_deploy`
Root causes to name: `api/cpu_exhaustion; redis/network_partition`

MEND AI'S INCIDENT REPORT

`redis/network_partition`: redis is not attached to the backend network it belongs on (list_services); 44 name-resolution errors in logs, e.g. probe /api/health: {"checks":{"cache":"error: Error -2 connecting to redis:6379. Name does not resolve.","database":"ok
`api/cpu_exhaustion`: 9 process(es) of 'python /app/tools/reindex_search.py' running beside the main service process (run_diagnostic processes); api container CPU at 48% (get_metrics); latency degraded: /api/orders/recent 698ms
Summary: [confirmed] re-attach redis to the backend network (restores its DNS name) (confidence 0.94; evidence: redis is not attached to the backend network it belongs on (list_services)). Prevent: monitor container network attachment and DNS resolution between services. | [confirmed] stop the runaway 'python /app/tools/reindex_search.py' job on api (pid 24 and its children) (confidence 0.76; evidence: 9 process(es) of 'python /app/tools/reindex_search.py' running beside the main service process (run_diagnostic processes)). Prevent: run batch jobs outside the serving container or under a CPU quota.

INVESTIGATION AND ACTIONS · 45 TOOL CALLS · 2 STATE CHANGES · 44.5 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.2s	OBSERVE	`check_endpoints()`
3	2.5s	OBSERVE	`get_recent_changes(limit=20)`
4	2.5s	OBSERVE	`get_metrics()`
5	6.1s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`
6	6.1s	OBSERVE	`get_logs(service=api, since_seconds=120, tail=400)`
7	6.1s	OBSERVE	`get_logs(service=worker, since_seconds=120, tail=400)`
8	6.2s	OBSERVE	`get_logs(service=postgres, since_seconds=120, tail=400)`
9	6.2s	OBSERVE	`get_logs(service=redis, since_seconds=120, tail=400)`
10	6.2s	OBSERVE	`run_diagnostic(service=gateway, check=upstream_http)`
11	6.8s	OBSERVE	`get_config(service=gateway)`

#	TIME	TYPE	TOOL CALL
12	6.8s	OBSERVE	`get_config(service=gateway, version=2, diff_against=1)`
13	6.8s	OBSERVE	`run_diagnostic(service=api, check=disk)`
14	7.9s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
15	8.0s	OBSERVE	`run_diagnostic(service=api, check=processes)`
16	9.6s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
17	9.7s	ACT	`reconnect_network(service=redis, network=backend)`
18	17.8s	OBSERVE	`list_services()`
19	18.0s	OBSERVE	`check_endpoints()`
20	19.9s	OBSERVE	`get_recent_changes(limit=20)`
21	19.9s	OBSERVE	`get_metrics()`
22	23.2s	OBSERVE	`get_logs(service=gateway, since_seconds=9, tail=400)`
23	23.2s	OBSERVE	`get_logs(service=api, since_seconds=9, tail=400)`
24	23.2s	OBSERVE	`get_logs(service=worker, since_seconds=9, tail=400)`
25	23.3s	OBSERVE	`get_logs(service=postgres, since_seconds=9, tail=400)`
26	23.3s	OBSERVE	`get_logs(service=redis, since_seconds=9, tail=400)`
27	23.3s	OBSERVE	`run_diagnostic(service=api, check=disk)`
28	24.4s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
29	24.5s	OBSERVE	`run_diagnostic(service=api, check=processes)`
30	26.6s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
31	26.7s	ACT	`kill_process(service=api, pid=24)`
32	32.8s	OBSERVE	`list_services()`
33	32.9s	OBSERVE	`check_endpoints()`
34	33.0s	OBSERVE	`get_recent_changes(limit=20)`
35	33.0s	OBSERVE	`get_metrics()`
36	35.3s	OBSERVE	`get_logs(service=gateway, since_seconds=8, tail=400)`
37	35.3s	OBSERVE	`get_logs(service=api, since_seconds=8, tail=400)`
38	35.3s	OBSERVE	`get_logs(service=worker, since_seconds=8, tail=400)`
39	35.4s	OBSERVE	`get_logs(service=postgres, since_seconds=8, tail=400)`
40	35.4s	OBSERVE	`get_logs(service=redis, since_seconds=8, tail=400)`
41	35.4s	OBSERVE	`run_diagnostic(service=api, check=disk)`
42	35.5s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
43	35.6s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
44	35.7s	OBSERVE	`verify_recovery()`

#	TIME	TYPE	TOOL CALL
45	44.5s	FINAL	`submit_report(root_causes=[{'component': 'redis', 'category': 'network_partition', 'evidence': 'redis is not attached to the backend network it belongs on (list_services); 44 name-resolution errors in logs, e.g. probe /api/health: {"checks":{"cache":"error: Error -2 connecting to redis:6379. Name does not resolve.","database":"ok'}, {'component': 'api', 'category': 'cpu_exhaustion', 'evidence': "9 process(es) of 'python /app/tools/reindex_search.py' running beside the main service process (run_diagnostic processes); api container CPU at 48% (get_metrics); latency degraded: /api/orders/recent 698ms"}], recovered=True, summary=[confirmed] re-attach redis to the backend network (restores its DNS name) (confidence 0.94; evidence: redis is not attached to the backend network it belongs on (list_services)). Prevent: monitor container network attachment and DNS resolution between services. | [confirmed] stop the runaway 'python /app/tools/reindex_search.py' job on api (pid 24 and its children) (confidence 0.76; evidence: 9 process(es) of 'python /app/tools/reindex_search.py' running beside the main service process (run_diagnostic processes)). Prevent: run batch jobs outside the serving container or under a CPU quota.)`

INDEPENDENT VERIFICATION

100% user traffic success	**25 ms** p95 latency	**0** collateral actions	**1.00** diagnosis F1

RECOMMENDATIONS

Immediate fix **DONE BY THE AGENT**

- Stop the runaway reindex_search job on api (kill_process), then confirm latency returns to normal.
- Reconnect redis to the backend network (reconnect_network), then confirm its dependents recover.

Prevent a repeat

- Run batch jobs like search reindexing on a worker or a schedule, never inside the serving api container.
- Set CPU limits per container so one job cannot starve request handling.
- Alert on sustained high CPU together with rising p95 latency.
- Manage network attachments only through docker compose or infrastructure code, never by hand.
- Add connectivity checks between dependent services to monitoring.
- Alert on dependency health (database/cache unreachable) with the exact component named.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

HELD-OUT

ho-10-cpu-and-redis-partition

SYSTEM RECOVERED

100 / 100

recovery 60 · diagnosis 30 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 80%, p95 693ms over the last few seconds.
  - GET /api/health: 2/2 failed, statuses [200] ({"checks":{"cache":"error: Error -2 connecting to
redis:6379. Name does not reso)
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`cpu_hog` on `api`: A runaway search reindex job saturates the API's CPU.
`network_partition` on `redis`: A backend service is cut off from the backend network.
Red herrings planted: `benign_gateway_deploy`
Root causes to name: `api/cpu_exhaustion; redis/network_partition`

MEND AI'S INCIDENT REPORT

`redis/network_partition`: redis is not attached to the backend network it belongs on (list_services); 44 name-resolution errors in logs, e.g. probe /api/health: {"checks":{"cache":"error: Error -2 connecting to redis:6379. Name does not resolve.","database":"ok
`api/cpu_exhaustion`: 9 process(es) of 'python /app/tools/reindex_search.py' running beside the main service process (run_diagnostic processes); api container CPU at 52% (get_metrics); latency degraded: /api/health 780ms
Summary: [confirmed] re-attach redis to the backend network (restores its DNS name) (confidence 0.94; evidence: redis is not attached to the backend network it belongs on (list_services)). Prevent: monitor container network attachment and DNS resolution between services. | [confirmed] stop the runaway 'python /app/tools/reindex_search.py' job on api (pid 23 and its children) (confidence 0.76; evidence: 9 process(es) of 'python /app/tools/reindex_search.py' running beside the main service process (run_diagnostic processes)). Prevent: run batch jobs outside the serving container or under a CPU quota.

INVESTIGATION AND ACTIONS · 45 TOOL CALLS · 2 STATE CHANGES · 44.1 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.1s	OBSERVE	`check_endpoints()`
3	1.9s	OBSERVE	`get_recent_changes(limit=20)`
4	1.9s	OBSERVE	`get_metrics()`
5	5.4s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`
6	5.4s	OBSERVE	`get_logs(service=api, since_seconds=120, tail=400)`
7	5.4s	OBSERVE	`get_logs(service=worker, since_seconds=120, tail=400)`
8	5.5s	OBSERVE	`get_logs(service=postgres, since_seconds=120, tail=400)`
9	5.5s	OBSERVE	`get_logs(service=redis, since_seconds=120, tail=400)`
10	5.5s	OBSERVE	`run_diagnostic(service=gateway, check=upstream_http)`
11	6.2s	OBSERVE	`get_config(service=gateway)`

#	TIME	TYPE	TOOL CALL
12	6.2s	OBSERVE	`get_config(service=gateway, version=2, diff_against=1)`
13	6.2s	OBSERVE	`run_diagnostic(service=api, check=disk)`
14	7.8s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
15	7.9s	OBSERVE	`run_diagnostic(service=api, check=processes)`
16	9.6s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
17	9.7s	ACT	`reconnect_network(service=redis, network=backend)`
18	17.7s	OBSERVE	`list_services()`
19	17.9s	OBSERVE	`check_endpoints()`
20	19.7s	OBSERVE	`get_recent_changes(limit=20)`
21	19.7s	OBSERVE	`get_metrics()`
22	23.5s	OBSERVE	`get_logs(service=gateway, since_seconds=9, tail=400)`
23	23.5s	OBSERVE	`get_logs(service=api, since_seconds=9, tail=400)`
24	23.5s	OBSERVE	`get_logs(service=worker, since_seconds=9, tail=400)`
25	23.6s	OBSERVE	`get_logs(service=postgres, since_seconds=9, tail=400)`
26	23.6s	OBSERVE	`get_logs(service=redis, since_seconds=9, tail=400)`
27	23.6s	OBSERVE	`run_diagnostic(service=api, check=disk)`
28	25.0s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
29	25.1s	OBSERVE	`run_diagnostic(service=api, check=processes)`
30	26.3s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
31	26.4s	ACT	`kill_process(service=api, pid=23)`
32	32.3s	OBSERVE	`list_services()`
33	32.4s	OBSERVE	`check_endpoints()`
34	32.5s	OBSERVE	`get_recent_changes(limit=20)`
35	32.5s	OBSERVE	`get_metrics()`
36	34.7s	OBSERVE	`get_logs(service=gateway, since_seconds=8, tail=400)`
37	34.8s	OBSERVE	`get_logs(service=api, since_seconds=8, tail=400)`
38	34.8s	OBSERVE	`get_logs(service=worker, since_seconds=8, tail=400)`
39	34.8s	OBSERVE	`get_logs(service=postgres, since_seconds=8, tail=400)`
40	34.8s	OBSERVE	`get_logs(service=redis, since_seconds=8, tail=400)`
41	34.9s	OBSERVE	`run_diagnostic(service=api, check=disk)`
42	35.0s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
43	35.0s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
44	35.1s	OBSERVE	`verify_recovery()`

#	TIME	TYPE	TOOL CALL
45	44.1s	FINAL	`submit_report(root_causes=[{'component': 'redis', 'category': 'network_partition', 'evidence': 'redis is not attached to the backend network it belongs on (list_services); 44 name-resolution errors in logs, e.g. probe /api/health: {"checks":{"cache":"error: Error -2 connecting to redis:6379. Name does not resolve.","database":"ok'}, {'component': 'api', 'category': 'cpu_exhaustion', 'evidence': "9 process(es) of 'python /app/tools/reindex_search.py' running beside the main service process (run_diagnostic processes); api container CPU at 52% (get_metrics); latency degraded: /api/health 780ms"}], recovered=True, summary=[confirmed] re-attach redis to the backend network (restores its DNS name) (confidence 0.94; evidence: redis is not attached to the backend network it belongs on (list_services)). Prevent: monitor container network attachment and DNS resolution between services. | [confirmed] stop the runaway 'python /app/tools/reindex_search.py' job on api (pid 23 and its children) (confidence 0.76; evidence: 9 process(es) of 'python /app/tools/reindex_search.py' running beside the main service process (run_diagnostic processes)). Prevent: run batch jobs outside the serving container or under a CPU quota.)`

INDEPENDENT VERIFICATION

100% user traffic success	**22 ms** p95 latency	**0** collateral actions	**1.00** diagnosis F1

RECOMMENDATIONS

Immediate fix **DONE BY THE AGENT**

- Stop the runaway reindex_search job on api (kill_process), then confirm latency returns to normal.
- Reconnect redis to the backend network (reconnect_network), then confirm its dependents recover.

Prevent a repeat

- Run batch jobs like search reindexing on a worker or a schedule, never inside the serving api container.
- Set CPU limits per container so one job cannot starve request handling.
- Alert on sustained high CPU together with rising p95 latency.
- Manage network attachments only through docker compose or infrastructure code, never by hand.
- Add connectivity checks between dependent services to monitoring.
- Alert on dependency health (database/cache unreachable) with the exact component named.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

HELD-OUT

ho-11-conn-leak-and-dns

SYSTEM RECOVERED

100 / 100

recovery 60 · diagnosis 30 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 40%, p95 6ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: [Errno -2]
Name does not resolve","di)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`conn_exhaustion` on `worker`: A backfill job in the worker leaks DB connections (and reconnects).
`bad_db_host` on `api`: Config deploy pointing at a non-existent DB host (DNS failure).
Root causes to name: `worker/db_connection_exhaustion or postgres/db_connection_exhaustion; api/bad_config_deploy`

MEND AI'S INCIDENT REPORT

`worker/db_connection_exhaustion`: postgres has 38/40 client connections (get_metrics); 35 'too many clients' errors, e.g. 2026-10-30 03:40:10.113 UTC [324] FATAL: remaining connection slots are reserved for roles with the SUPERUSER attribute; application 'report-backfill' holds 37 sessions (run_diagnostic db_activity)
`api/bad_config_deploy`: 36 log line(s) on api: a hostname does not resolve, e.g. probe /api/health: {"checks":{"cache":"ok","database":"error: [Errno -2] Name does not resolve","disk":"ok"},"status":"; v2 changed host (get_config diff), matching the errors; api was deployed 0.6 min ago: v2 'point api at new primary database endpoint' by ci-bot
Summary: [confirmed] stop 'python /app/tools/report_backfill.py' on worker (it keeps reconnecting), then terminate the 'report-backfill' database sessions to free connection slots (confidence 0.85; evidence: postgres has 38/40 client connections (get_metrics)). Prevent: use a connection pool with a per-client cap and alert on connection saturation. | [confirmed] roll back the api config to v1 (undoing v2) (confidence 0.76; evidence: 36 log line(s) on api: a hostname does not resolve, e.g. probe /api/health: {"checks":{"cache":"ok","database":"error: [Errno -2] Name does not resolve","disk":"ok"},"status":"). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.

INVESTIGATION AND ACTIONS · 48 TOOL CALLS · 3 STATE CHANGES · 41.9 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.1s	OBSERVE	`check_endpoints()`
3	0.1s	OBSERVE	`get_recent_changes(limit=20)`
4	0.1s	OBSERVE	`get_metrics()`
5	2.4s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`
6	2.5s	OBSERVE	`get_logs(service=api, since_seconds=120, tail=400)`
7	2.5s	OBSERVE	`get_logs(service=worker, since_seconds=120, tail=400)`

#	TIME	TYPE	TOOL CALL
8	2.5s	OBSERVE	get_logs(service=postgres, since_seconds=120, tail=400)
9	2.5s	OBSERVE	get_logs(service=redis, since_seconds=120, tail=400)
10	2.6s	OBSERVE	get_config(service=api)
11	2.6s	OBSERVE	get_config(service=api, version=2, diff_against=1)
12	2.6s	OBSERVE	run_diagnostic(service=api, check=disk)
13	2.7s	OBSERVE	run_diagnostic(service=worker, check=disk)
14	2.8s	OBSERVE	run_diagnostic(service=postgres, check=db_locks)
15	2.9s	OBSERVE	run_diagnostic(service=postgres, check=db_activity)
16	3.1s	OBSERVE	run_diagnostic(service=worker, check=processes)
17	3.3s	ACT	kill_process(service=worker, pid=7)
18	3.4s	ACT	terminate_db_sessions(application_name=report-backfill)
19	8.5s	OBSERVE	list_services()
20	8.6s	OBSERVE	check_endpoints()
21	8.7s	OBSERVE	get_recent_changes(limit=20)
22	8.7s	OBSERVE	get_metrics()
23	10.9s	OBSERVE	get_logs(service=gateway, since_seconds=8, tail=400)
24	10.9s	OBSERVE	get_logs(service=api, since_seconds=8, tail=400)
25	11.0s	OBSERVE	get_logs(service=worker, since_seconds=8, tail=400)
26	11.0s	OBSERVE	get_logs(service=postgres, since_seconds=8, tail=400)
27	11.0s	OBSERVE	get_logs(service=redis, since_seconds=8, tail=400)
28	11.1s	OBSERVE	get_config(service=api)
29	11.1s	OBSERVE	get_config(service=api, version=2, diff_against=1)
30	11.1s	OBSERVE	run_diagnostic(service=api, check=disk)
31	11.2s	OBSERVE	run_diagnostic(service=worker, check=disk)
32	11.3s	OBSERVE	run_diagnostic(service=postgres, check=db_locks)
33	11.4s	OBSERVE	run_diagnostic(service=postgres, check=db_activity)
34	11.6s	ACT	rollback_config(service=api, to_version=1)
35	30.1s	OBSERVE	list_services()
36	30.3s	OBSERVE	check_endpoints()
37	30.3s	OBSERVE	get_recent_changes(limit=20)
38	30.3s	OBSERVE	get_metrics()
39	32.6s	OBSERVE	get_logs(service=gateway, since_seconds=9, tail=400)
40	32.7s	OBSERVE	get_logs(service=api, since_seconds=9, tail=400)
41	32.7s	OBSERVE	get_logs(service=worker, since_seconds=9, tail=400)

#	TIME	TYPE	TOOL CALL
42	32.7s	OBSERVE	`get_logs(service=postgres, since_seconds=9, tail=400)`
43	32.8s	OBSERVE	`get_logs(service=redis, since_seconds=9, tail=400)`
44	32.8s	OBSERVE	`run_diagnostic(service=api, check=disk)`
45	32.9s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
46	33.0s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
47	33.1s	OBSERVE	`verify_recovery()`
48	41.9s	FINAL	`submit_report(root_causes=[{'component': 'worker', 'category': 'db_connection_exhaustion', 'evidence': "postgres has 38/40 client connections (get_metrics); 35 'too many clients' errors, e.g. 2026-10-30 03:40:10.113 UTC [324] FATAL: remaining connection slots are reserved for roles with the SUPERUSER attribute; application 'report-backfill' holds 37 sessions (run_diagnostic db_activity)"}, {'component': 'api', 'category': 'bad_config_deploy', 'evidence': '36 log line(s) on api: a hostname does not resolve, e.g. probe /api/health: {"checks":{"cache":"ok","database":"error: [Errno -2] Name does not resolve","disk":"ok"},"status":"; v2 changed host (get_config diff), matching the errors; api was deployed 0.6 min ago: v2 \'point api at new primary database endpoint\' by ci-bot'}], recovered=True, summary=[confirmed] stop 'python /app/tools/report_backfill.py' on worker (it keeps reconnecting), then terminate the 'report-backfill' database sessions to free connection slots (confidence 0.85; evidence: postgres has 38/40 client connections (get_metrics)). Prevent: use a connection pool with a per-client cap and alert on connection saturation. | [confirmed] roll back the api config to v1 (undoing v2) (confidence 0.76; evidence: 36 log line(s) on api: a hostname does not resolve, e.g. probe /api/health: {"checks":{"cache":"ok","database":"error: [Errno -2] Name does not resolve","disk":"ok"},"status":"). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.)`

INDEPENDENT VERIFICATION

100% user traffic success	**25 ms** p95 latency	**0** collateral actions	**1.00** diagnosis F1

RECOMMENDATIONS

Immediate fix **DONE BY THE AGENT**

- Stop the leaking report_backfill job in the worker and terminate its database sessions.
- Roll back the api config to the last version that points at the reachable database host (postgres).

Prevent a repeat

- Use a bounded connection pool in every job, and close connections in finally blocks.
- Give each application its own connection limit (per-role CONNECTION LIMIT or a pooler such as PgBouncer).
- Alert when database connections pass 80% of max_connections.
- Validate that every hostname in a config change resolves from inside the target environment before deploying.
- Change database endpoints behind a stable DNS alias, not by editing each service's config.
- Roll back automatically when the dependency health check fails right after a deploy.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

HELD-OUT

ho-11-conn-leak-and-dns

SYSTEM RECOVERED

100 / 100

recovery 60 · diagnosis 30 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 40%, p95 6ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [503] ({"checks":{"cache":"ok","database":"error: [Errno -2]
Name does not resolve","di)
  - GET /api/orders/recent: 5/5 failed, statuses [503] ({"error":"database error"}
)
  - POST /api/orders: 5/5 failed, statuses [503] ({"error":"database error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`conn_exhaustion` on `worker`: A backfill job in the worker leaks DB connections (and reconnects).
`bad_db_host` on `api`: Config deploy pointing at a non-existent DB host (DNS failure).
Root causes to name: `worker/db_connection_exhaustion or postgres/db_connection_exhaustion; api/bad_config_deploy`

MEND AI'S INCIDENT REPORT

`worker/db_connection_exhaustion`: postgres has 38/40 client connections (get_metrics); 35 'too many clients' errors, e.g. 2026-10-30 04:01:56.943 UTC [313] FATAL: remaining connection slots are reserved for roles with the SUPERUSER attribute; application 'report-backfill' holds 37 sessions (run_diagnostic db_activity)
`api/bad_config_deploy`: 36 log line(s) on api: a hostname does not resolve, e.g. probe /api/health: {"checks":{"cache":"ok","database":"error: [Errno -2] Name does not resolve","disk":"ok"},"status":"; v2 changed host (get_config diff), matching the errors; api was deployed 0.6 min ago: v2 'point api at new primary database endpoint' by ci-bot
Summary: [confirmed] stop 'python /app/tools/report_backfill.py' on worker (it keeps reconnecting), then terminate the 'report-backfill' database sessions to free connection slots (confidence 0.85; evidence: postgres has 38/40 client connections (get_metrics)). Prevent: use a connection pool with a per-client cap and alert on connection saturation. | [confirmed] roll back the api config to v1 (undoing v2) (confidence 0.76; evidence: 36 log line(s) on api: a hostname does not resolve, e.g. probe /api/health: {"checks":{"cache":"ok","database":"error: [Errno -2] Name does not resolve","disk":"ok"},"status":"). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.

INVESTIGATION AND ACTIONS · 48 TOOL CALLS · 3 STATE CHANGES · 42.0 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.1s	OBSERVE	`check_endpoints()`
3	0.2s	OBSERVE	`get_recent_changes(limit=20)`
4	0.2s	OBSERVE	`get_metrics()`
5	2.5s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`
6	2.5s	OBSERVE	`get_logs(service=api, since_seconds=120, tail=400)`
7	2.5s	OBSERVE	`get_logs(service=worker, since_seconds=120, tail=400)`

#	TIME	TYPE	TOOL CALL
8	2.5s	OBSERVE	get_logs(service=postgres, since_seconds=120, tail=400)
9	2.6s	OBSERVE	get_logs(service=redis, since_seconds=120, tail=400)
10	2.6s	OBSERVE	get_config(service=api)
11	2.6s	OBSERVE	get_config(service=api, version=2, diff_against=1)
12	2.6s	OBSERVE	run_diagnostic(service=api, check=disk)
13	2.7s	OBSERVE	run_diagnostic(service=worker, check=disk)
14	2.8s	OBSERVE	run_diagnostic(service=postgres, check=db_locks)
15	2.9s	OBSERVE	run_diagnostic(service=postgres, check=db_activity)
16	3.1s	OBSERVE	run_diagnostic(service=worker, check=processes)
17	3.3s	ACT	kill_process(service=worker, pid=7)
18	3.4s	ACT	terminate_db_sessions(application_name=report-backfill)
19	8.5s	OBSERVE	list_services()
20	8.6s	OBSERVE	check_endpoints()
21	8.6s	OBSERVE	get_recent_changes(limit=20)
22	8.6s	OBSERVE	get_metrics()
23	10.9s	OBSERVE	get_logs(service=gateway, since_seconds=8, tail=400)
24	10.9s	OBSERVE	get_logs(service=api, since_seconds=8, tail=400)
25	11.0s	OBSERVE	get_logs(service=worker, since_seconds=8, tail=400)
26	11.0s	OBSERVE	get_logs(service=postgres, since_seconds=8, tail=400)
27	11.0s	OBSERVE	get_logs(service=redis, since_seconds=8, tail=400)
28	11.1s	OBSERVE	get_config(service=api)
29	11.1s	OBSERVE	get_config(service=api, version=2, diff_against=1)
30	11.1s	OBSERVE	run_diagnostic(service=api, check=disk)
31	11.2s	OBSERVE	run_diagnostic(service=worker, check=disk)
32	11.3s	OBSERVE	run_diagnostic(service=postgres, check=db_locks)
33	11.4s	OBSERVE	run_diagnostic(service=postgres, check=db_activity)
34	11.6s	ACT	rollback_config(service=api, to_version=1)
35	30.1s	OBSERVE	list_services()
36	30.2s	OBSERVE	check_endpoints()
37	30.3s	OBSERVE	get_recent_changes(limit=20)
38	30.3s	OBSERVE	get_metrics()
39	32.7s	OBSERVE	get_logs(service=gateway, since_seconds=9, tail=400)
40	32.7s	OBSERVE	get_logs(service=api, since_seconds=9, tail=400)
41	32.7s	OBSERVE	get_logs(service=worker, since_seconds=9, tail=400)

#	TIME	TYPE	TOOL CALL
42	32.8s	OBSERVE	`get_logs(service=postgres, since_seconds=9, tail=400)`
43	32.8s	OBSERVE	`get_logs(service=redis, since_seconds=9, tail=400)`
44	32.8s	OBSERVE	`run_diagnostic(service=api, check=disk)`
45	33.0s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
46	33.0s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
47	33.1s	OBSERVE	`verify_recovery()`
48	42.0s	FINAL	`submit_report(root_causes=[{'component': 'worker', 'category': 'db_connection_exhaustion', 'evidence': "postgres has 38/40 client connections (get_metrics); 35 'too many clients' errors, e.g. 2026-10-30 04:01:56.943 UTC [313] FATAL: remaining connection slots are reserved for roles with the SUPERUSER attribute; application 'report-backfill' holds 37 sessions (run_diagnostic db_activity)"}, {'component': 'api', 'category': 'bad_config_deploy', 'evidence': '36 log line(s) on api: a hostname does not resolve, e.g. probe /api/health: {"checks":{"cache":"ok","database":"error: [Errno -2] Name does not resolve","disk":"ok"},"status":"; v2 changed host (get_config diff), matching the errors; api was deployed 0.6 min ago: v2 \'point api at new primary database endpoint\' by ci-bot'}], recovered=True, summary=[confirmed] stop 'python /app/tools/report_backfill.py' on worker (it keeps reconnecting), then terminate the 'report-backfill' database sessions to free connection slots (confidence 0.85; evidence: postgres has 38/40 client connections (get_metrics)). Prevent: use a connection pool with a per-client cap and alert on connection saturation. | [confirmed] roll back the api config to v1 (undoing v2) (confidence 0.76; evidence: 36 log line(s) on api: a hostname does not resolve, e.g. probe /api/health: {"checks":{"cache":"ok","database":"error: [Errno -2] Name does not resolve","disk":"ok"},"status":"). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.)`

INDEPENDENT VERIFICATION

100% user traffic success	**23 ms** p95 latency	**0** collateral actions	**1.00** diagnosis F1

RECOMMENDATIONS

Immediate fix **DONE BY THE AGENT**

- Stop the leaking report_backfill job in the worker and terminate its database sessions.
- Roll back the api config to the last version that points at the reachable database host (postgres).

Prevent a repeat

- Use a bounded connection pool in every job, and close connections in finally blocks.
- Give each application its own connection limit (per-role CONNECTION LIMIT or a pooler such as PgBouncer).
- Alert when database connections pass 80% of max_connections.
- Validate that every hostname in a config change resolves from inside the target environment before deploying.
- Change database endpoints behind a stable DNS alias, not by editing each service's config.
- Roll back automatically when the dependency health check fails right after a deploy.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

HELD-OUT

ho-12-triple

SYSTEM RECOVERED

94 / 100

recovery 60 · diagnosis 24 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 60%, p95 22ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [200] ({"checks":{"cache":"error: Error -2 connecting to
redis:6379. Name does not reso)
  - POST /api/orders: 5/5 failed, statuses [500] ({"error":"pricing error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`service_stopped` on `redis`: A core service process is stopped.
`disk_full` on `api`: A runaway export fills the API data volume.
`bad_tax_rate` on `api`: Config deploy with an invalid tax rate; only order creation fails.
Root causes to name: `redis/service_down; api/disk_full; api/bad_config_deploy`

MEND AI'S INCIDENT REPORT

`redis/service_down`: redis container is exited (list_services); 2 endpoint(s) failing while redis is down
`api/bad_config_deploy`: 14 log line(s) on api: the service rejected its own configuration, e.g. 2026-10-30T03:42:24 ERROR api pricing config invalid: could not convert string to float: '15%'; v2 changed 15% (get_config diff), matching the errors; api was deployed 0.6 min ago: v2 'update VAT rate for new fiscal year' by finance-config-ui

Summary: [confirmed] start the redis container (confidence 0.93; evidence: redis container is exited (list_services)). Prevent: add a restart policy and alert on container exits. I [confirmed] roll back the api config to v1 (undoing v2) (confidence 0.76; evidence: 14 log line(s) on api: the service rejected its own configuration, e.g. 2026-10-30T03:42:24 ERROR api pricing config invalid: could not convert string to float: '15%'). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.

INVESTIGATION AND ACTIONS · 44 TOOL CALLS · 2 STATE CHANGES · 44.2 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.1s	OBSERVE	`check_endpoints()`
3	0.2s	OBSERVE	`get_recent_changes(limit=20)`
4	0.2s	OBSERVE	`get_metrics()`
5	2.5s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`
6	2.5s	OBSERVE	`get_logs(service=api, since_seconds=120, tail=400)`
7	2.5s	OBSERVE	`get_logs(service=worker, since_seconds=120, tail=400)`
8	2.6s	OBSERVE	`get_logs(service=postgres, since_seconds=120, tail=400)`
9	2.6s	OBSERVE	`get_logs(service=redis, since_seconds=120, tail=400)`
10	2.6s	OBSERVE	`get_config(service=api)`
11	2.6s	OBSERVE	`get_config(service=api, version=2, diff_against=1)`

#	TIME	TYPE	TOOL CALL
12	2.6s	OBSERVE	run_diagnostic(service=api, check=disk)
13	2.7s	OBSERVE	run_diagnostic(service=worker, check=disk)
14	2.8s	OBSERVE	run_diagnostic(service=postgres, check=db_locks)
15	2.9s	ACT	start_service(service=redis)
16	11.2s	OBSERVE	list_services()
17	11.3s	OBSERVE	check_endpoints()
18	11.4s	OBSERVE	get_recent_changes(limit=20)
19	11.4s	OBSERVE	get_metrics()
20	13.6s	OBSERVE	get_logs(service=gateway, since_seconds=9, tail=400)
21	13.7s	OBSERVE	get_logs(service=api, since_seconds=9, tail=400)
22	13.7s	OBSERVE	get_logs(service=worker, since_seconds=9, tail=400)
23	13.7s	OBSERVE	get_logs(service=postgres, since_seconds=9, tail=400)
24	13.7s	OBSERVE	get_logs(service=redis, since_seconds=9, tail=400)
25	13.8s	OBSERVE	get_config(service=api)
26	13.8s	OBSERVE	get_config(service=api, version=2, diff_against=1)
27	13.8s	OBSERVE	run_diagnostic(service=api, check=disk)
28	13.8s	OBSERVE	run_diagnostic(service=worker, check=disk)
29	14.0s	OBSERVE	run_diagnostic(service=postgres, check=db_locks)
30	14.1s	ACT	rollback_config(service=api, to_version=1)
31	32.6s	OBSERVE	list_services()
32	32.7s	OBSERVE	check_endpoints()
33	32.8s	OBSERVE	get_recent_changes(limit=20)
34	32.8s	OBSERVE	get_metrics()
35	35.1s	OBSERVE	get_logs(service=gateway, since_seconds=9, tail=400)
36	35.1s	OBSERVE	get_logs(service=api, since_seconds=9, tail=400)
37	35.1s	OBSERVE	get_logs(service=worker, since_seconds=9, tail=400)
38	35.1s	OBSERVE	get_logs(service=postgres, since_seconds=9, tail=400)
39	35.2s	OBSERVE	get_logs(service=redis, since_seconds=9, tail=400)
40	35.2s	OBSERVE	run_diagnostic(service=api, check=disk)
41	35.3s	OBSERVE	run_diagnostic(service=worker, check=disk)
42	35.4s	OBSERVE	run_diagnostic(service=postgres, check=db_locks)
43	35.5s	OBSERVE	verify_recovery()

MEND

#	TIME	TYPE	TOOL CALL
44	44.2s	FINAL	`submit_report(root_causes=[{'component': 'redis', 'category': 'service_down', 'evidence': 'redis container is exited (list_services); 2 endpoint(s) failing while redis is down'}, {'component': 'api', 'category': 'bad_config_deploy', 'evidence': "14 log line(s) on api: the service rejected its own configuration, e.g. 2026-10-30T03:42:24 ERROR api pricing config invalid: could not convert string to float: '15%'; v2 changed 15% (get_config diff), matching the errors; api was deployed 0.6 min ago: v2 'update VAT rate for new fiscal year' by finance-config-ui"}], recovered=True, summary=[confirmed] start the redis container (confidence 0.93; evidence: redis container is exited (list_services)). Prevent: add a restart policy and alert on container exits. | [confirmed] roll back the api config to v1 (undoing v2) (confidence 0.76; evidence: 14 log line(s) on api: the service rejected its own configuration, e.g. 2026-10-30T03:42:24 ERROR api pricing config invalid: could not convert string to float: '15%'). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.)`

INDEPENDENT VERIFICATION

100% user traffic success	**21 ms** p95 latency	**0** collateral actions	**0.80** diagnosis F1

RECOMMENDATIONS

Immediate fix **DONE BY THE AGENT**

- Start the redis service (start_service), then confirm the endpoints recover.
- Delete the runaway temporary export under /data/tmp/ on api, then confirm /data is below 80%.
- Roll back the api config to the last version with a numeric tax rate, then confirm order creation works.

Prevent a repeat

- Set a restart policy (restart: unless-stopped) so a stopped container comes back on its own.
- Alert on service state and container exits, not only on failing user traffic.
- Record who stops production services and require a change ticket for manual stops.
- Alert when the data volume passes 80% so it is caught before writes fail.
- Write exports to object storage or a separate volume with a quota, not to the app's data volume.
- Clean up temporary files automatically (TTL) and cap export sizes.
- Type-check business settings (tax rate must be a number between 0 and 1) in the config UI and in CI.
- Monitor each user journey separately: here only order creation failed while health checks looked fine.
- Require review for finance-owned config changes that reach production.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.

HELD-OUT

ho-12-triple

SYSTEM RECOVERED

94 / 100

recovery 60 · diagnosis 24 · safety 10

PAGER ALERT THE AGENT RECEIVED

```
PAGE [SEV2] ShopStack synthetic monitoring is failing.
Probe success 60%, p95 19ms over the last few seconds.
  - GET /api/health: 5/5 failed, statuses [200] ({"checks":{"cache":"error: Error -2 connecting to
redis:6379. Name does not reso)
  - POST /api/orders: 5/5 failed, statuses [500] ({"error":"pricing error"}
)
Investigate, restore service, verify recovery, then submit an incident report.
```

WHAT WAS ACTUALLY BROKEN (HIDDEN FROM THE AGENT)

`service_stopped` on `redis`: A core service process is stopped.
`disk_full` on `api`: A runaway export fills the API data volume.
`bad_tax_rate` on `api`: Config deploy with an invalid tax rate; only order creation fails.
Root causes to name: `redis/service_down; api/disk_full; api/bad_config_deploy`

MEND AI'S INCIDENT REPORT

`redis/service_down`: redis container is exited (list_services); 2 endpoint(s) failing while redis is down
`api/bad_config_deploy`: 16 log line(s) on api: the service rejected its own configuration, e.g. 2026-10-30T04:04:10 ERROR api pricing config invalid: could not convert string to float: '15%'; v2 changed 15% (get_config diff), matching the errors; api was deployed 0.6 min ago: v2 'update VAT rate for new fiscal year' by finance-config-ui
Summary: [confirmed] start the redis container (confidence 0.93; evidence: redis container is exited (list_services)). Prevent: add a restart policy and alert on container exits. I [confirmed] roll back the api config to v1 (undoing v2) (confidence 0.76; evidence: 16 log line(s) on api: the service rejected its own configuration, e.g. 2026-10-30T04:04:10 ERROR api pricing config invalid: could not convert string to float: '15%'). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.

INVESTIGATION AND ACTIONS · 44 TOOL CALLS · 2 STATE CHANGES · 44.4 S

#	TIME	TYPE	TOOL CALL
1	0.0s	OBSERVE	`list_services()`
2	0.1s	OBSERVE	`check_endpoints()`
3	0.2s	OBSERVE	`get_recent_changes(limit=20)`
4	0.2s	OBSERVE	`get_metrics()`
5	2.5s	OBSERVE	`get_logs(service=gateway, since_seconds=120, tail=400)`
6	2.5s	OBSERVE	`get_logs(service=api, since_seconds=120, tail=400)`
7	2.5s	OBSERVE	`get_logs(service=worker, since_seconds=120, tail=400)`
8	2.6s	OBSERVE	`get_logs(service=postgres, since_seconds=120, tail=400)`
9	2.6s	OBSERVE	`get_logs(service=redis, since_seconds=120, tail=400)`
10	2.6s	OBSERVE	`get_config(service=api)`
11	2.6s	OBSERVE	`get_config(service=api, version=2, diff_against=1)`

#	TIME	TYPE	TOOL CALL
12	2.6s	OBSERVE	`run_diagnostic(service=api, check=disk)`
13	2.8s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
14	2.9s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
15	3.0s	ACT	`start_service(service=redis)`
16	11.3s	OBSERVE	`list_services()`
17	11.4s	OBSERVE	`check_endpoints()`
18	11.5s	OBSERVE	`get_recent_changes(limit=20)`
19	11.5s	OBSERVE	`get_metrics()`
20	13.8s	OBSERVE	`get_logs(service=gateway, since_seconds=9, tail=400)`
21	13.8s	OBSERVE	`get_logs(service=api, since_seconds=9, tail=400)`
22	13.8s	OBSERVE	`get_logs(service=worker, since_seconds=9, tail=400)`
23	13.8s	OBSERVE	`get_logs(service=postgres, since_seconds=9, tail=400)`
24	13.9s	OBSERVE	`get_logs(service=redis, since_seconds=9, tail=400)`
25	13.9s	OBSERVE	`get_config(service=api)`
26	13.9s	OBSERVE	`get_config(service=api, version=2, diff_against=1)`
27	13.9s	OBSERVE	`run_diagnostic(service=api, check=disk)`
28	14.0s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
29	14.1s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
30	14.2s	ACT	`rollback_config(service=api, to_version=1)`
31	32.8s	OBSERVE	`list_services()`
32	32.9s	OBSERVE	`check_endpoints()`
33	33.0s	OBSERVE	`get_recent_changes(limit=20)`
34	33.0s	OBSERVE	`get_metrics()`
35	35.2s	OBSERVE	`get_logs(service=gateway, since_seconds=9, tail=400)`
36	35.3s	OBSERVE	`get_logs(service=api, since_seconds=9, tail=400)`
37	35.3s	OBSERVE	`get_logs(service=worker, since_seconds=9, tail=400)`
38	35.3s	OBSERVE	`get_logs(service=postgres, since_seconds=9, tail=400)`
39	35.3s	OBSERVE	`get_logs(service=redis, since_seconds=9, tail=400)`
40	35.4s	OBSERVE	`run_diagnostic(service=api, check=disk)`
41	35.4s	OBSERVE	`run_diagnostic(service=worker, check=disk)`
42	35.5s	OBSERVE	`run_diagnostic(service=postgres, check=db_locks)`
43	35.6s	OBSERVE	`verify_recovery()`

MEND

#	TIME	TYPE	TOOL CALL
44	44.4s	FINAL	`submit_report(root_causes=[{'component': 'redis', 'category': 'service_down', 'evidence': 'redis container is exited (list_services); 2 endpoint(s) failing while redis is down'}, {'component': 'api', 'category': 'bad_config_deploy', 'evidence': "16 log line(s) on api: the service rejected its own configuration, e.g. 2026-10-30T04:04:10 ERROR api pricing config invalid: could not convert string to float: '15%'; v2 changed 15% (get_config diff), matching the errors; api was deployed 0.6 min ago: v2 'update VAT rate for new fiscal year' by finance-config-ui"}], recovered=True, summary=[confirmed] start the redis container (confidence 0.93; evidence: redis container is exited (list_services)). Prevent: add a restart policy and alert on container exits. | [confirmed] roll back the api config to v1 (undoing v2) (confidence 0.76; evidence: 16 log line(s) on api: the service rejected its own configuration, e.g. 2026-10-30T04:04:10 ERROR api pricing config invalid: could not convert string to float: '15%'). Prevent: validate configs in CI and canary deploys with automatic rollback on error-rate spikes.)`

INDEPENDENT VERIFICATION

100% user traffic success	**22 ms** p95 latency	**0** collateral actions	**0.80** diagnosis F1

RECOMMENDATIONS

Immediate fix **DONE BY THE AGENT**

- Start the redis service (start_service), then confirm the endpoints recover.
- Delete the runaway temporary export under /data/tmp/ on api, then confirm /data is below 80%.
- Roll back the api config to the last version with a numeric tax rate, then confirm order creation works.

Prevent a repeat

- Set a restart policy (restart: unless-stopped) so a stopped container comes back on its own.
- Alert on service state and container exits, not only on failing user traffic.
- Record who stops production services and require a change ticket for manual stops.
- Alert when the data volume passes 80% so it is caught before writes fail.
- Write exports to object storage or a separate volume with a quota, not to the app's data volume.
- Clean up temporary files automatically (TTL) and cap export sizes.
- Type-check business settings (tax rate must be a number between 0 and 1) in the config UI and in CI.
- Monitor each user journey separately: here only order creation failed while health checks looked fine.
- Require review for finance-owned config changes that reach production.

Fix and prevention advice come from the remediation playbook for the injected fault types, not from the agent.